



BRNO UNIVERSITY OF TECHNOLOGY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF COMPUTER GRAPHICS AND MULTIMEDIA

ÚSTAV POČÍTAČOVÉ GRAFIKY A MULTIMÉDIÍ

GENERATING CODE FROM TEXTUAL DESCRIPTION OF FUNCTIONALITY

GENEROVÁNÍ KÓDU Z TEXTOVÉHO POPISU FUNKCIONALITY

BACHELOR'S THESIS

BAKALÁŘSKÁ PRÁCE

AUTHOR

AUTOR PRÁCE

ONDREJ KOŽÁNYI

SUPERVISOR

VEDOUCÍ PRÁCE

doc. RNDr. PAVEL SMRŽ, Ph.D.

BRNO 2025

Bachelor's Thesis Assignment



162801

Institut: Department of Computer Graphics and Multimedia (DCGM)
Student: **Kožányi Ondrej**
Programme: Information Technology
Title: **Generating Code from Textual Description of Functionality**
Category: Information Systems
Academic year: 2024/25

Assignment:

1. Get to know how to use large-scale language models and how to adapt them to domain data, such as GitHub and StackExchange.
2. Experiment with fine-tuning of chosen models on collected data to improve model performance.
3. Explore agentic approaches to implementing AI-based systems, mainly the emergent reasoning capabilities.
4. Based on the knowledge gained, design and implement a multi-agent system that can analyse the user's codebase and perform various code-related tasks (such as test generation, documentation...) assigned by user queries.
5. Evaluate the results of the system on a representative sample of data, discussing the dependence of the results on the intuitive complexity of the description.
6. Prepare a poster presenting the work, its goals and results.

Literature:

- according to the supervisor's suggestion

Requirements for the semestral defence:

- a functional prototype of the system

Detailed formal requirements can be found at <https://www.fit.vut.cz/study/theses/>

Supervisor: **Smrž Pavel, doc. RNDr., Ph.D.**
Head of Department: Černocký Jan, prof. Dr. Ing.
Beginning of work: 1.11.2024
Submission deadline: 14.5.2025
Approval date: 7.5.2025

Abstract

The increasing complexity of software development often causes developers to be overrun with routine, yet essential code-related tasks, such as adding simple new features, debugging, writing tests, and documenting code, taking away from precious time reserved for high-level decisions and actions. To address these concerns, this thesis presents the exploration, design, and implementation of CogniCollab, a modular multi-agent Artificial Intelligence assistant system, focused on automating typical routine tasks that developers encounter. CogniCollab utilizes Large Language Models as foundational building blocks for the agents and features an Analyst Agent responsible for natural language query processing, task decomposition, and planning further actions, alongside Specialized Agents which handle sub-task completion. The system is enriched by the integration and adaptation of the LocAgent tool, capable of performing semantic searches over a provided codebase, enhancing contextual understanding, and the utilization of a fine-tuned CodeBERT model, which detects intent classes from the original assignments. Agents are each responsible for a separate part of the development with specialized roles and iterative reasoning logic. The system's ability to successfully assist in day-to-day software development tasks is demonstrated through practical scenarios showcasing CogniCollab's two modes and a comparative discussion with other state-of-the-art systems, highlighting the importance of agentic methods in AI assistant development.

Abstrakt

Narastajúca zložitost vývoja softvéru často spôsobuje, že vývojári sú zahltení rutinnými, no nevyhnutnými úlohami súvisiacimi s kódom, ako je pridávanie jednoduchých nových funkcií, ladenie, písanie testov a dokumentovanie kódu, čo im ubera drahocenný čas vyhradený pre rozhodnutia a činnosti na vyššej úrovni. Na riešenie týchto problémov táto práca predstavuje prieskum, návrh a implementáciu systému CogniCollab, modulárneho multiagentného asistenta umelej inteligencie, zameraného na automatizáciu typických rutinných úloh, s ktorými sa vývojári stretávajú. CogniCollab využíva rozsiahle jazykové modely (LLM) ako základné stavebné prvky pre agentov a zahŕňa analyzačného agenta (Analyst Agent) zodpovedného za spracovanie dopytov v prirodzenom jazyku, dekompozíciu úloh a plánovanie ďalších akcií, popri špecializovaných agentoch (Specialized Agents), ktorí zabezpečujú splnenie podúloh. Systém je obohatený o integráciu a adaptáciu nástroja LocAgent, schopného vykonávať sémantické vyhľadávanie v poskytnutej kódovej báze, čím zlepšuje kontextuálne porozumenie, a využitie doladeného modelu CodeBERT, ktorý deteguje triedy zámerov z pôvodných zadaní. Každý agent je zodpovedný za samostatnú časť vývoja so špecializovanými rolami a logikou iteračného uvažovania. Schopnosť systému úspešne asistovať pri každodenných úlohách vývoja softvéru je demonštrovaná prostredníctvom praktických scenárov predstavujúcich dva režimy systému CogniCollab a porovnávacou diskusiou s inými najmodernejšími systémami, čo zdôrazňuje dôležitosť agentových metód vo vývoji AI asistentov.

Keywords

large language models, Transformer architecture, AI agents, assistant systems, software development automatization, multi-agent collaboration, role definition, code generation, prompt engineering, context awareness

Klíčová slova

velké jazykové modely, Transformer architektúra, agenti umelej inteligencie, asistenčné systémy, automatizácia vývoja softvéru, multiagentná spolupráca, definícia rolí, generovanie kódu, inžinierstvo promptov, povedomie o kontexte

Reference

KOŽÁNYI, Ondrej. *Generating Code from Textual Description of Functionality*. Brno, 2025. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor doc. RNDr. Pavel Smrž, Ph.D.

Rozšířený abstrakt

Moderný svet vývoja softvéru je charakterizovaný vzrástajúcou komplexitou, spôsobujúcou, že vývojári sú často zahltení rutinnými, no dôležitými úlohami spojenými s generovaním alebo upravovaním kódu. Medzi tieto úlohy patrí napríklad pridávanie relatívne jednoduchých funkcionalít, ladenie nefunkčného kódu, vytváranie jednotkových testov alebo písanie dokumentácie. Repetívny charakter rutinných úloh z nich robí ideálneho kandidáta pre automatizáciu, za účelom šetrenia drahocenného času vývojárov, ktorý môže byť následne využitý na architekturné rozhodnutia vyššej úrovne. Táto bakalárska práca sa zaoberá týmito výzvami a jej hlavným cieľom je preskúmať, navrhnúť a implementovať modulárny, multi-agentný systém založený na umelej inteligencii schopný autonómne riešiť zadané úlohy, týkajúce sa vývoja stanoveného lokálneho repozitára. Tento nástroj s názvom CogniCollab je určený na ulahčenie bežného života programátorov poskytovaním pomoci s vývojom bez nutnosti manuálneho zadávania kontextu projektu pri každej interakcii. Hypotézou práce je, že implementácia systémov využívajúcich agentné metódy uvažovania, plánovania, používania nástrojov a kolaboratívneho multiagentného prostredia môže efektívne adresovať neefektívnosť vo vývoji softvéru.

Systém CogniCollab je postavený na modulárnej multiagentnej architektúre, kde každý agent má zadanú špecifikovanú detailnú rolu pomocou dôkladného návrhu promptov, ktoré definujú agentove povinnosti a operačné požiadavky. Základným stavebným kameňom agentov sú veľké jazykové modely (LLM), poskytujúce prominentné inteligenčné schopnosti. Podstatnú postavu systému predstavuje Analyzačný Agent (Analyst Agent), ktorý funguje ako centrálny nadriadený, interagujúci s vývojárom. Jeho povinnosti zahŕňajú pochopenie zadanej úlohy, dekompozícia na menšie, zvládnuteľné podproblémy a následné plánovanie akcií potrebných na ich vyriešenie. Jeho operácia je zakončená konsolidáciou výsledkov a vytvorením finálnej odpovede vývojárovi. Analyzačný Agent poskytuje dva operačné režimy: klarifikačný a dokončovací. Tieto režimy reprezentujú typické scenáre práce programátorov kedy potrebujú buď vysvetliť existujúci, nejasný kód (klarifikačný režim), alebo realizovať špecifické vývojárske úlohy (dokončovací režim).

Dôležitou súčasťou tejto práce je aj integrovaný a adaptovaný externý systém LocAgent. Nástroj bol pôvodne vyvinutý na lokalizáciu kódu vo verzovaných repositároch na platforme GitHub, na základe textu voľne zadaného v prirodzenom jazyku užívateľom, no bol upravený na prácu s lokálne uloženými repozitármi. Tento systém je schopný vykonať hĺbkovú analýzu poskytnutej implementácie projektu, vytvoriť jej grafovú reprezentáciu a následne vykonávať sémantické vyhľadávanie relevantných segmentov kódu. Týmto spôsobom poskytuje kontextuálnu pomoc nielen Analyzačnému Agentovi (analýzou kódovej báze pri prvotnom obdržaní úlohy), ale aj ostatným agentom, ktorí môžu využívať vyhľadávacie funkcie LocAgent systému. Toto zabezpečuje, že CogniCollab je schopný spracovávať aj veľké projekty s množstvom súborov.

Pred samostatnou plánovacou fázou Analyzačného Agentu prechádza požiadavka užívateľa cez proces klasifikácie zámerov. Na realizovanie tejto časti systému bol použitý predtrénovaný CodeBERT model, ďalej doladený na identifikáciu preddefinovaných tried zámeru (`write_tests`, `debug_code`, `add_feature`, `document_code`) z voľne zadaných vývojárskych úloh. Identifikované triedy následne informujú Analyzačného Agentu o tom, ktorí špecializovaní agenti budú pravdepodobne potrební na splnenie zadania.

Za vykonanie samotných podúloh, delegovaných Analyzačným Agentom, sú zodpovední Špecializovaní Agenti, pričom každý zodpovedá za separátnu časť vývoja (Implementačný Agent, Ladiaci Agent, Testovací Agent, Dokumentačný Agent). Na vyriešenie úloh využívajú iteratívne rozmýšľanie, typicky podľa slučky "Myšlienka – Akcia – Pozorovanie", a na

získanie potrebného contextu kódu môžu využiť funkcie nástroja LocAgent. Komunikáciu systémových komponentov uchováva zdieľaná pamäťová štruktúra a je riadená centrálnym Komunikačným Vodcom. Tento vodca preposiela obdržané správy (zadania a výsledky podúloh) jednotlivým agentom a sleduje aktuálny stav riešenia. Celý nástroj obaluje prívetivé užívateľské rozhranie, zabezpečujúce jednoduchú interakciu so systémom.

Dosiahnuté schopnosti systému CogniCollab boli demonštrované pomocou dvoch praktických scenárov zodpovedajúcim operačným režimom Analyzačného Agentu. V klariфикаčnom scenári bol CogniCollab schopný vyprodukovať vhodné vysvetlenia účelu a fungovania špecifickej časti kódu na základe abstraktných otázok vývojára, čím preukázal svoje schopnosti porozumenia kódu a poskytnutia relevantných informácií. V dokončovacom scenári systém úspešne vyriešil komplexnejšiu úlohu zahŕňajúcu niekoľko rôznych činností. Vygenerovaním požadovanej funkcionality, sprevádzanej odpovedajúcou dokumentáciou a jednotkovými testami, CogniCollab preukázal efektívnu spoluprácu medzi Analyzačným Agentom a Špecializovanými Agentmi. Toto zhodnotenie výsledkov je doplnené o diskusiu zameranú na podobnosti a odlišnosti nástroja CogniCollab s inými uznávanými implementáciami AI asistentov. Zhodnotením týchto prác sa ukázalo, že kým multi-agentné systémy ako MetaGPT a SoA sa primárne sústreďia na generovanie nových projektov z abstraktných zadanií, CogniCollab, podobne ako Aider, sa orientuje na vylepšovanie už existujúcich projektov. Oproti typickým párovým programátorom však CogniCollab využíva tím špecializovaných agentov a pokročilé mechanizmy pre kontextové porozumenie (LocAgent), čo mu umožňuje riešiť úlohy s vyššou mierou autonómie a porozumenia.

Hlavné prínosy nástroja CogniCollab do oblasti vývoja softvéru zahŕňajú schopnosť automatizovať rutinné programátorské úlohy, dosiahnutie vyššieho levelu abstrakcie, týkajúcej sa prístupu systému k relevantnému existujúcemu kódu, výrazné zlepšenie kontextuálneho pochopenia prostredníctvom integrovaného nástroja LocAgent a modulárnu architektúru, umožňujúcu jednoduché zaradenie nových špecializovaných agentov. Popri týchto prínosoch, CogniCollab, vo svojom súčasnom stave vývoja, je taktiež postihovaný rôznymi obmedzeniami. Primárne limitácie súvisia s prevádzkovou latenciou systému, ktorá je zväčša spôsobená využívaním bezplatných API kľúčov s obmedzenou priepustnosťou, a aspektami tolerancie chýb kde sú agenti pri neočakávaných podmienkach zahltení, čo môže negatívne ovplyvniť kvalitu a celkovú kompletnosť výsledkov.

Táto práca úspešne demonštrovala dôležitosť a inovatívnosť agentných prístupov prostredníctvom výskumu, návrhu a realizácie systému CogniCollab, ktorý osvojuje tieto prístupy na celkové uľahčenie a urýchlenie vývoja. Metódy ako plánovanie, dekompozícia, iteratívne uvažovanie, využívanie nástrojov a multi-agentná spolupráca sa javia kľúčovými pre ďalší pokrok v oblasti inteligentných asistentov. Práca mierená na CogniCollab v budúcnosti by mohla potenciálne zahŕňať zrýchlenie operačného procesu systému využitím spoľahlivejších poskytovateľov API prístupu k modelom, vylepšenie robustnosti alebo celkové rozšírenie funkcionality definovaním nových špecializovaných rôľ.

Generating Code from Textual Description of Functionality

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of doc. RNDr. Pavel Smrž, Ph.D. I have listed all the literary sources, publications and other sources, which were used during the preparation of this thesis.

.....
Ondrej Kožányi
14.5.2025

Acknowledgements

I would like to express my deepest gratitude to the supervisor of my Bachelor's thesis doc. RNDr. Pavel Smrž, Ph.D., for his valuable guidance, patience, and active help throughout working on this paper.

Contents

1	Introduction	4
2	Large Language Models	6
2.1	History	6
2.2	Notable Models	10
3	Agents powered by LLMs	14
3.1	Connectionist vs. Symbolic AI	14
3.2	LLM-powered Agent Capabilities	15
3.3	Considerations	21
4	Problem Specification and System Design	23
4.1	Problem specification	23
4.2	System Design	27
5	Implementation Details	31
5.1	Environment and External Tools	31
5.2	LLM Interaction	34
5.3	System Orchestration	35
5.4	Analyst Agent	36
5.5	Intent Classification	37
5.6	Agent Execution	39
5.7	Communication Infrastructure	40
5.8	Specialized Agents	41
5.9	Prompt Engineering	42
6	Results Assessment	44
6.1	System Demonstrations	44
6.2	CogniCollab in the Context of State of the Art AI-driven systems	48
6.3	System Limitations	50
7	Conclusion	52
7.1	Summary of Key Contributions	52
7.2	CogniCollab Significance and Limitations Acknowledgement	53
7.3	Future Work	54
	Bibliography	56
A	Contents of the included storage media	62

List of Figures

2.1	Transformer encoded-decoder architecture. Taken from the foundational work [59]	8
2.2	The two phases of training Large Language Models.	9
3.1	This figure displays the principle of Chain-of-Thought prompting. An example solution for a similar problem is included in the prompt to encourage the model’s reasoning. Figure taken from [61]	17
3.2	ReAct Paradigm Adaptation	20
3.3	High-level architecture of an LLM empowered agent with its three main components. Figure is taken from [65].	21
4.1	Architecture overview of the multi-agent system and its message flow.	28
5.1	Overview of the LocAgent framework. The codebase is first parsed to create a graph-based representation. Then, LocAgent constructs sparse indexes that allow exploring the project structures and searching code context. Figure taken from [12].	32
5.2	CogniCollab employs principles of RAG through the integrated LocAgent, which retrieves necessary information from the specified codebase.	34
5.3	Depiction of the upsampling strategy, which balances out the dataset across classes by augmenting underrepresented ones.	38
6.1	The final response of the Analyst Agent’s clarification mode, formatted and displayed in the Streamlit user interface.	45
6.2	Left: Scenario for a single agent completing code completion tasks. As the complexity and assignment scale increase, the agent is limited by its maximum context length, preventing it from completing the task. Right: SoA’s dynamic agent instantiation, which evenly divides the assignment across agents, ensuring a constant workload. Figure taken from [29].	49

Chapter 1

Introduction

The development of scalable, robust, and maintainable software remains at the heart of technological progress. However, the process itself is riddled with inherent complexities. In modern development, software often comprises vast codebases and nuanced dependencies, and it requires collaboration across teams of developers, raising the efforts necessary for efficient management. Developers often encounter a straining workload of routine coding tasks, such as writing documentation, generating unit tests, performing basic code optimizations, and identifying and fixing simple bugs. These tasks are, of course, necessary for ensuring code quality throughout the development, although they significantly reduce the time available for higher-value activities, including decisions pertaining to architectural design, expanding software applications, or other tasks requiring complex problem-solving capabilities. Hindering development with accumulated routine tasks can ultimately impact productivity and cause project delays. Additionally, maintaining consistency in coding standards and documentation over the whole project is an ongoing challenge. Current tools frequently utilize a lower level of abstraction and struggle to capture the crucial context as the codebase size increases, thus making them unsuitable for large-scale development.

Recent breakthroughs in the field of Artificial Intelligence (AI), namely the advancements made with Large Language Models (LLMs) utilizing the novel Transformer [59] architecture, have enabled the emergence of revolutionary capabilities in understanding and generating natural language and structured code. These capabilities offered new possibilities for automating segments of the software development process. However, since development complexity varies considerably based on the project size, the properties of sole LLMs might not suffice. Their true potential for radical improvement possibly lies in extending the original design of LLMs, limited to single-turn interactions, into the field of *agentic AI implementations*. Agentic systems utilize LLMs as the main building block (the „brain“ of an agent) and encapsulate them in a specialized control structure, allowing for explicit reasoning, greater autonomy, and a goal-directed approach. The hypothesis of this thesis is that implementing systems leveraging agentic methods of reasoning, planning, self-reflection, tool use, and a collaborative multi-agent environment can effectively mitigate the inefficiencies in software development by automating routine tasks. This is achieved with a high level of abstraction, specifically by enhancing contextual understanding within existing large-scale codebases, consequently reducing the user’s time spent on manually providing the necessary context, and by providing a user-friendly interface, encapsulating system operations.

This thesis focuses on exploring the possible applications of agentic AI systems for streamlining software implementation. It begins by reviewing the technologies foundational

to these systems, the capabilities of the novel LLMs, underlined by its Transformer [59] architecture, and highlights the features of innovative models. Follows the exploration of core concepts and techniques that make AI autonomous agents possible, such as planning strategies, refinement loops, tool calling, and collaboration approaches. Subsequently, this work analyses the inherent issues in software development, emphasizing the potential for automating them, and reviews the current limitations of existing agentic frameworks.

Building upon this foundation, the main aim and contribution of this work is to specify, design, and implement a modular, multi-agent assistant tool for the automation of routine coding tasks, CogniCollab (representing Cognitive agents Collaborating to complete a higher objective). The architecture of this system highlights scalability through the possibility of easily adding new agents and a higher level of abstraction by allowing the users to input instructions in natural language. Context awareness limitations are especially addressed through the integration of a sophisticated codebase analysis mechanism and code search-and-retrieval tool. Specifically, CogniCollab incorporates an *Analyst Agent* augmented with an external code localization tool, LocAgent, capable of creating a graph representation of the codebase and retrieving code segments based on the agent’s search, enabling it to find and reason over necessary information, even in complex projects. To effectively complete assigned objectives, this system employs multiple specialized agents with divided labor (e.g., documentation, testing, bug fixing), coordinated by a central communication leader, which routes sub-tasks and results to the corresponding target agents, and a shared memory structure for effective message passing and dependency management. Inspirations for this design came from the aspects and general nature of human development team collaboration.

The system facilitates a workflow beginning with natural language assignment from the developer, continues through intent classification, codebase analysis, and task decomposition performed by the Analyst Agent, iterative completion of tasks by specialized agents utilizing reasoning loops, and concludes by collecting the individual results and presenting them back to the user. This aims to abstract the developer from the process of completing elementary coding tasks, freeing their time for higher-level objectives.

This thesis will conclude with an assessment of the developed system’s capabilities through a qualitative study of selected workflow scenarios and discussions about correlations and differences with various remarkable AI assistant systems, followed by an overview of future development ideas for CogniCollab.

Chapter 2

Large Language Models

This chapter will review Large Language Models as the core technologies that make autonomous AI agents possible.

Large Language Models (LLMs) are a scientific breakthrough in the field of artificial intelligence, especially when it comes to natural language processing (NLP) and code generation [10]. This combination of abilities makes them adept at handling casually structured tasks given by a user interacting with the model.

2.1 History

The need for processing sequential data was an increasingly urgent issue during the early stages of NLP. Much later, the development of Recurrent Neural Networks (RNNs) provided a robust mechanism for this, by maintaining a hidden state that captures information from previous inputs [39].

2.1.1 Recurrent Neural Networks

The concept of RNNs dates back to the 1980s, with foundational models like the Jordan network [32] (1986) and the Elman network [17] (1990). The proposal included feedback loops that enabled capturing temporal dynamics data. To achieve modeling sequences over time, the network needed to maintain a form of memory realized by feeding the outputs back into the network.

Training these early RNNs proved more difficult, however, due to issues such as vanishing and exploding gradients [46]. This inhibits the process of learning long-term dependencies.

To further RNNs and resolve the met challenges, **Long Short-Term Memory** (LSTM) network was introduced in 1997. LSTM utilized gating mechanisms to regulate the flow of information, which resulted in effectively capturing long-range dependencies and reducing gradient hindrances [22].

To offer a more streamlined and simple architecture as an alternative to LSTM, **Gated Recurrent Unit** (GRU) [13] was introduced, with comparable performance to the original LSTM.

The RNNs were successfully applied to various sequence modeling tasks. The main contributions were observed in including language modeling, machine translation, speech recognition, and time-series prediction [39]. By solving the issue of processing sequential data, they played a vital part in the development of more advanced models.

However, sequential processing of data meant that RNNs were limited in terms of parallelization and efficiency. The later-introduced Transformer architecture marked a significant shift, by enabling models to process sequences in parallel, leading to improved performance and scalability.

2.1.2 Transformers

The Transformer architecture [59] revolutionized the field by employing *Self-Attention mechanism*.

Self-Attention The traditional attention mechanism used in previous architectures ensures models dynamically focus on the most relevant parts of the input sequence when generating each element of the output sequence [7]. Translation is a good example. When generating the next word of the translated output, the model attends to that word of the original sequence that is currently being processed. Here, attention is from one sequence to another — the output depends those parts of the input that are most relevant at each step.

In contrast, self-attention enables models to capture relationships between different parts of a single input sequence [59]. When processing the input, each token is processed in relevance to all other tokens in the sequence. This ensures that the model captures dependencies regardless of the distance of tokens within the sequence. This is very important, especially for tasks aiming to understand the meaning of a sentence — like for classification or summarization [15].

Single-head attention captures only a limited range of dependencies. To counter this, each layer of the encoder/decoder stack employs a multi-head attention block (which consists of multiple attention heads), enhancing its ability to model complex relationships within sequences [59].

Positional Encoding Unlike previous, recurrent models, Transformers process all tokens in a sequence simultaneously and therefore lack the positional knowledge of the tokens [59]. This effectively means, that without a method to address this, the model would not differentiate between the two sequences „cat jumped on the tree“ and „jumped tree cat the on“. Positional encoding ensures that information about the positions of tokens is incorporated into the input embeddings, enabling the model to capture the order of the sequence.

Architecture

The Transformer architecture consists of two main components – *encoder* and *decoder* [59].

- **Encoder** The encoder in this architecture is a stack of identical layers, each processing the input with a multi-head self-attention mechanism and a feed-forward neural network [24] [59]. The encoder processes the sequence simultaneously, and its purpose is to capture contextual relationships between tokens. Positional encodings are used here to preserve token position awareness.
- **Decoder** The decoder contains multiple stacked layers as well, but includes an additional attention mechanism to focus on the encoder’s output [59]. It utilizes a masked self-attention mechanism to ensure that each prediction is conditioned on previously generated tokens, preventing the model from peeking at future tokens during training.

There are three variants of this architecture. **Encoder-only** models incorporate the Transformer’s encoder stack to process input and generate contextually aware representations. The representations can then be used for tasks like classification, sentiment analysis, or named entity recognition. Their goal is usually not to generate new sequences of text. Included in this category of models is BERT, employing bidirectional attention to capture context from both preceding and succeeding tokens [15]. The decoder component of the architecture allows **decoder-only** models to generate text in an autoregressive manner, predicting the next token based on previous ones. They excel at tasks involving text generation, such as language modeling, story generation, or code completion. Decoder-only models are becoming more popular with the emergence of AI chat systems, where the model generates coherent and contextually relevant text sequences. The very popular GPT model series adopts this architecture [50]. Models utilizing both components of the Transformer, **encoder-decoder** models, such as T5 [51] and BART [37], find their use in tasks that require transforming an input sequence into a different output sequence, like machine translation or summarization. The encoder component processes the entire input and constructs a comprehensive representation, which is then used by the decoder to produce the output sequence, attending to the encoder’s output at each step. The most effective use of this variant is in situations when the output depends heavily on the input context.

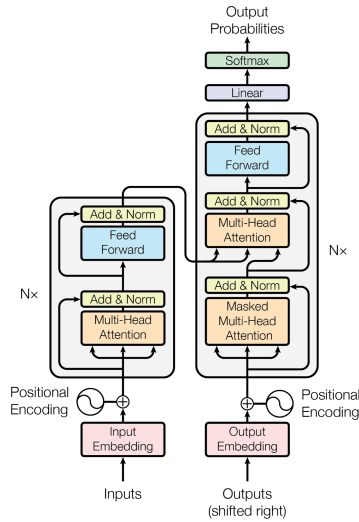


Figure 2.1: Transformer encoded-decoder architecture. Taken from the foundational work [59]

Pre-training and Fine-tuning Paradigm

The model’s training process is divided into two stages: *pre-training* and *fine-tuning* [40].

1. **Pre-Training:** In this first stage, the model is trained using a large corpus of data in a self-supervised way of learning. The goal is to predict tokens following a given input. The specific training details may vary based on the model’s architecture (e.g. encoder-decoder, decoder-only).
2. **Fine-Tuning:** This stage aims to further adapt a model for a specific problem. Below are different approaches to fine-tuning.

- **Transfer Learning:** The large corpus of data enables pre-trained models to perform well for various general tasks. Models meant to handle downstream (specific, application-oriented) tasks must be fine-tuned using this task-specific data from the problem domain. This method is called Transfer Learning.
- **Instruction-tuning:** Models can be fine-tuned using instruction-formatted data, meaning instruction and an input-output pair [69]. Instruction is generally a multi-task data described in natural language, meant to guide the model in generating an answer based on the prompt and input. This process improves the quality of generated responses to user queries.
- **Alignment-tuning:** Using human feedback to align the models during fine-tuning, we can mitigate generating false, biased, or harmful text. This can involve prompting the model to generate unstable responses and then modifying the model's parameters to avoid inconsistencies. This ensures the model behaves according to human expectations. A proven method for model alignment is reinforcement learning with human feedback (RLHF) [45], applying a combination of *Reward modeling* and *Reinforcement learning*.

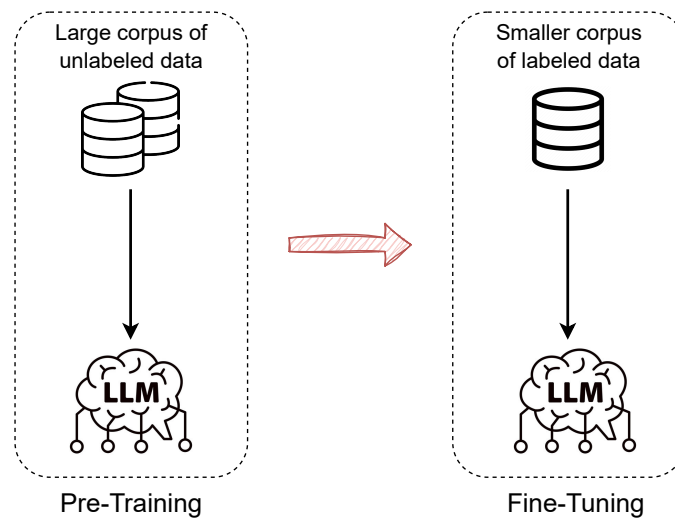


Figure 2.2: The two phases of training Large Language Models.

This paradigm aids the automation of software engineering tasks. Pre-training the models on large datasets injects a deep understanding of programming languages and data structures. Successfully pre-trained models can be fine-tuned for diverse software development tasks, like code completion or test generation, with more specific data [44].

Fine-tuning a pre-trained model requires less data and computational resources compared to attempting to train the model from scratch. This is important in the context of software engineering because obtaining large amounts of labeled data may be expensive or complicated in real-world scenarios [11].

Models can be fine-tuned on multiple related tasks simultaneously, encouraging the development of flexible tools that are suitable for assisting with different aspects of software development [4]. Multi-task learning enables the model to generalize when presented with

unseen cases and adapt to different tasks. Conversely, fine-tuning can be used to narrow the model’s focus on a specific task or domain. This reduces the risk of confusing the model with unrelated knowledge and improves precision for domain-specific tasks.

Benefits for AI assistant systems

Since Transformer models process the input sequences in parallel, modern hardware (e.g., GPUs) can be effectively utilized, facilitating the training of larger models on extensive datasets [59]. The ability to process vast amounts of information while preserving time and computational resources is crucial for agentic AI systems to make fast decisions.

By analysing the evidence gained from experimental findings, we can see that expanding the model size by increasing the number of parameters and diversifying data during training effectively boosts their performance across various tasks [33]. It is frequently the case that as the model grows, it exhibits new abilities, such as enhanced reasoning and deeper understanding of nuanced contextual information. Similar prospects offer a significant boost to the autonomy and flexibility of agentic AI assistants.

The architecture’s scalability properties ensure the possibility of fine-tuning the Transformer models for a wide variety of tasks, spanning from natural language processing to computer vision [35]. This diversity is especially valuable for agentic systems, in which the agents usually possess specific roles and serve multiple functions.

Furthermore, the modularity offered by the Transformer models naturally complements the principles of agentic AI system design – the agents function as individual modules (system components) and can be fine-tuned, customized, or even replaced independently based on the specific roles needed in the system [63].

2.2 Notable Models

In this section, three exceptional models that heavily contribute to the field of AI will be reviewed. First, the BERT model, a member of encoder-only LLMs, followed by GPT-3 and GPT-4 underlined by their decoder-only architecture.

2.2.1 The BERT model

BERT (Bidirectional Encoder Representations from Transformers) [15] is an encoder-only model. It is distinctively characterized by its use of bidirectional context during training.

Unlike unidirectional models, which predict tokens using only past context, BERT conditions its token predictions on both the preceding and succeeding tokens. This means that the model encompasses the context of a word in relation to its surrounding words (left and also right), deepening the model’s contextual understanding.

Bidirectionality in this model is achieved by training with these two pre-training objectives:

- **Masked Language Model (MLM):** Randomly masking tokens in an input sequence. The model is then expected to predict the masked tokens based on their context. This deepens the bidirectional understanding of text, as the model needs to attend to context on both sides of the masked token.
- **Next Sentence Prediction (NSP):** In NLP, many downstream tasks like Question Answering (QA) rely on understanding the relationships between sentences. To help

BERT understand these relationships, we use a training task to predict whether one sentence logically follows another. Data used for this training is constructed from sentence pairs (sentence A and sentence B) where 50% of the time, sentence B does logically follow A, and 50% of the time, it is a random sentence from the corpus.

Thanks to its deep contextual awareness, BERT is a suitable model for code-oriented tasks like documentation and detailed analysis, where precise contextual interpretation is necessary. This model can play a vital role in agentic AI systems. For example, an autonomous documentation agent utilizing BERT might be included in the system to generate comprehensive documentation by analysing codebases, understanding function intentions, and leveraging contextual clues from comments and code structure.

2.2.2 The GPT models

This subsection introduces two significant decoder-only models, GPT-3 and GPT-4, performing especially well on generative tasks.

GPT-3

GPT-3 (Generative Pre-trained Transformer 3) was a breakthrough in the field of LLM development. With up to 175 billion parameters, it was considered the largest model at the time of its release. The ingenuity of this model lies in how it performs on a wide variety of NLP tasks with minimal to no task-specific training (zero-shot and few-shot learning) [10].

- *zero-shot learning*: The model is not given any fine-tuning, task-specific examples. The model relies on the general knowledge gained from pre-training.
- *few-shot learning*: The model is given a few examples within the prompt to guide its responses.

Having GPT-3 perform well even with little specific training demonstrates the significance of the model's size and pre-training on very large datasets with general knowledge.

Positive results were achieved on various NLP benchmarks, such as text generation, language translation, question answering, and arithmetic reasoning [10]. Handling specialized tasks such as programming code generation, poetry writing, and even solving logic puzzles proved effective as well, demonstrating the model's versatility.

The size of this LLM allows it to understand complex patterns and dependencies even with a larger input [10]. This ensures scalability to complex tasks involving large-scale repository codebases.

Implications for Agentic AI Systems GPT-3, exhibiting profound abilities in natural language understanding and text generation, offers a strong foundation for autonomous agents that must interpret, generate, or translate complex natural language instructions [56, 48]. Since it can generalize across tasks through prompt-based learning, the agent's responsibilities can be dynamically altered as needed without retraining. This is very important in agentic AI systems, where flexibility and rapid adaptation are crucial for tasks like natural language understanding, documentation, or user support automation.

2.2.3 GPT-4

GPT-4 offers significant improvements over its predecessors. GPT-4 is multimodal – it can process both text and image inputs while generating text outputs. This property substantially broadens its applicability across various domains [3].

Key Features

- **Multimodal Capabilities** As already mentioned, GPT-4 can condition its responses on both textual and visual inputs. This facilitates advanced tasks, such as image captioning and visual question answering.
- **Enhanced Performance** GPT-4 has proven capable on various professional and academic benchmarks with human-level performance. In an experiment simulating a bar exam, the model scored around the top 10% of test takers [34]. Conversely, GPT-3 placed at around the bottom 10% in that experiment. Therefore, GPT-4 demonstrates an enormous improvement over its predecessor.
- **Extended Context Window** This model incorporates a context window length of 32,768 tokens [3], and therefore surpasses GPT-3 (with a context window of 2,048 tokens in its original implementation) by a landslide. This leads to more coherent and contextually aware responses, over longer conversations and document processing.
- **Improved Alignment and Safety** By utilizing RLHF, GPT-4 displays signs of advanced adherence to user intentions and is less likely to output disallowed content, making it more reliable for sensitive applications [3].

Implications for Agentic AI Systems The new and improved capabilities of GPT-4 are even more suitable for the development of autonomous agents than the already powerful model, GPT-3 [55, 48]. Its profound proficiency in coding tasks, summarization, and complex problem-solving allows for seamless agent workflow with minimal human intervention. Furthermore, the multimodal property of this model offers the possibility of covering more domains and areas that would benefit from AI automation.

2.2.4 Considerations

As transformative as Transformer-based models like BERT and GPT are, their deployment and development involve several challenges that must be addressed. These challenges fall into two primary categories: computational and timely costs and ethical considerations.

Computational and Timely Costs

Training Transformer-based models, especially large-scale ones like GPT-3, requires significant computational resources and time:

- **Hardware Requirements:** Advanced hardware is needed to train LLMs, including hundreds of GPUs or TPUs running in parallel. GPT-3 was trained with a supercomputing infrastructure that handles trillions of floating-point operations per second (FLOPS) [10].
- **Energy Consumption:** The energy requirements for training large models have an environmental impact with high CO₂ emissions [8].

- **Time Costs:** Training models on massive datasets can take weeks or even months [14]. Once pre-trained, fine-tuning can be faster but still requires expenditure in careful hyperparameter tuning and validation.
- **Inference Costs:** After models are trained, running inference on them (sending input and generating output) requires substantial computational power. This puts a strain on real-time applications, making them costly and potentially lowering end-user satisfaction.

Ethical Considerations

The widespread adoption of Transformer-based models also raises several ethical concerns:

- **Bias in Generated Text:** Large language models often inherit biases from the datasets they are trained on. For example, if the training corpus contains biased or discriminatory content, the model is likely to replicate and even amplify these biases in its outputs [1]. Techniques like RLHF [45] are helpful in reducing biased outputs by using human feedback to guide the model during training.
- **Misinformation and Harmful Outputs:** Models may generate plausible but factually incorrect or harmful content [38]. Remember that models mainly predict the next most probable sequences of text, and that can sometimes go wrong.
- **Privacy Risks:** Training on large, unfiltered datasets can inadvertently expose private information [5] not meant for the model to see. Training must comply with privacy regulations (like GDPR—General Data Protection Regulation) to protect sensitive data.

Chapter 3

Agents powered by LLMs

The significant advancements in the field of Large Language models and language processing in general have facilitated not only the development of powerful general-purpose models but also the creation of intelligent autonomous systems [60]. Individual LLMs, proficient in tasks involving, for example, answering questions, summarizing text, or generating code, are limited by their prompt-response type of inference to single-turn interactions.

Conversely, utilizing agentic methods leads to greater autonomy [48]. The agent represents a control structure with an LLM as its foundation. This wrap of logic enables autonomous decisions through human-like practices, including task decomposition, iterative reasoning, reflection over achieved results, and more. Agents with properties such as this are goal-directed, stateful, and often operate iteratively in dynamic environments, conditioning their reasoning on past observations and internal knowledge.

3.1 Connectionist vs. Symbolic AI

Autonomous agents powered by LLMs represent a convergence of symbolic and connectionist paradigms in AI [65]. Unlike *connectionist AI*, which relies on learning connections and patterns from large data corpora, *symbolic AI* uses logic and explicit rules for reasoning. LLMs themselves exhibit some implicit signs of symbolic AI behavior (e.g., logic, planning, arithmetic) [61] while using connectionist infrastructure consisting of large neural networks trained on chunks of data. This brings the two approaches a little closer, and agentic capabilities introduce explicit symbolic AI elements on top of the LLMs' implicit reasoning.

Debates between connectionist and symbolic AI have often characterized the evolution of artificial intelligence. This section explores the differences between these two paradigms. The information presented here is derived from [65].

3.1.1 Historical Divide

The two paradigms were historically perceived as mutually exclusive, dividing the research into two groups. Some believe that symbolic AI represents the true essence of intelligence because it relies on formal logic, rule-based systems, and explicit knowledge representations that mirror human reasoning processes. Others argued that connectionist AI, inspired by biological neural networks, offers a more realistic model of human cognition by learning patterns and behaviors directly from data.

- **Symbolic AI:**

- Emphasizes explicit knowledge representation through rules, logic, and ontologies.
- Well-suited for tasks requiring formal reasoning and precise control, such as theorem proving or expert systems.
- Challenges include difficulty scaling to unstructured or noisy data and the rigidity of rule-based systems.

- **Connectionist AI:**

- Focuses on learning from examples through distributed representations in neural networks.
- Excels in pattern recognition and adaptability, as demonstrated in vision and natural language tasks.
- Historically criticized for lacking interpretability and explicit reasoning mechanisms.

3.1.2 Convergence through LLMs

As already discussed, LLMs represent the first step in bridging the gaps between the two paradigms:

- **Implicit Symbolic Reasoning:** LLMs can handle symbols (e.g., language tokens) in a manner that mirrors symbolic reasoning. They use mechanisms like self-attention [59] to capture relationships, deduce patterns, and generate logical outputs, even without explicit programming of rules.
- **Connectionist Foundation:** Built on neural network architectures, LLMs leverage vast amounts of data to learn representations, allowing them to generalize across diverse tasks without needing task-specific training [10]. The knowledge acquired in pre-training is useful to agentic design patterns, such as reflection, because they possess broad contextual understanding and the ability to reason across domains.

With these two building blocks, LLMs create the perfect opportunity to integrate symbolic AI behavior even further. Agentic methods fully close the gap by imposing explicit thinking capabilities within a structured logical framework [67], defining specific roles, and even facilitating inter-agent collaboration [23].

3.2 LLM-powered Agent Capabilities

This section will shed light on the key capabilities of AI-empowered agents. We will talk about *self-reflection*, *planning*, *tool-use*, and *multi-agent collaboration*. Unlike traditional, linear interactions between users and AI, agentic workflows facilitate a dynamic, iterative relationship, enabling systems to autonomously adapt and increase the quality of task resolutions [67, 54].

3.2.1 Self-Reflection

Classic LLMs often suffer from outputting incorrect or partially incorrect answers, which could be improved with a couple more prompts for the model. Self-reflection acts as a

feedback loop that enhances performance and reduces errors without the need for external intervention or model retraining. In other words, this automates the middle step of delivering critical feedback to the agent to get a better response. This principle is explored in depth through the Reflexion [54] framework, which employs verbal reinforcement learning, and the SELF-REFINE [41] approach, which iteratively improves outputs through self-feedback. These two frameworks approach self-reflection in a different way, but both reach the shared goal of the agent’s output improvement.

Reflexion framework

The key principle of this framework lies in providing the agent with verbal feedback derived from its task interaction.

It utilizes a dual-memory approach while storing the feedback. Short-term memory stores the context of recent tasks, while long-term memory contains lessons from past experiences. This behavior mimics the human learning process. Same as the agent, we focus on the task at hand while drawing knowledge from long-term experiences, balancing immediate feedback with generalized knowledge.

Reflexion comprises three components with different objectives. **Actor** – an instance of an LLM that is used to execute actions and generate desired text. The authors of this framework tried different generative LLMs in their experiments to explore the most suitable generative capabilities. **Evaluator** – appraises output generated by the Actor. It computes a reward score that reflects its performance within the given task context. Given the different nature of various tasks, it is sometimes difficult to define reward functions, so variations of this model were explored (e.g., reward based on exact match for decision-making tasks). **Self-Reflection** – generates verbal self-reflections to provide valuable feedback for future trials. It takes as input a sparse reward signal (e.g., success/fail), current trajectory, and the long-term memory and outputs detailed feedback, which is then stored in long-term memory. This way, when the agent is completing a similar task in the future, it accounts for this feedback and might take a different trajectory.

SELF-REFINE framework

This approach also aims to improve LLM outputs through self-feedback/reflection, but goes in a slightly different direction. While Reflexion works towards improving the process over time, SELF-REFINE incorporates an interactive refinement of the output. This approach is inspired by how people revise and improve their written text. After the model outputs its first answer, that same LLM is instructed to generate feedback on that answer. This feedback is then passed to the LLM with the following prompt and used by the model to refine its original solution. The refinement loop can continue as long as needed to achieve the desired quality.

The two frameworks showcase that Self-reflection can be implemented in different ways depending on the intended functionality and the properties we want to emphasize during the agent’s development.

3.2.2 Planning

Standalone LLMs are proficient in single-turn interactions but often struggle with more complex tasks that require more structure and foresight for their solution [60].

Enhancing LLMs with the ability to break down a complex assignment into manageable sub-problems and reason over the procedure over multiple iterations improves the results substantially. By equipping the agent with a structured approach to problem-solving, it is more likely to maintain context and gradually progress towards completing the ultimate goal.

Task Decomposition At the core of this capability lies task decomposition – dividing the complex, nuanced user request into structured steps that are easier to handle [63]. This can again be observed in human agent teams. Very few people start working on a complex task before drafting a step-by-step plan first, even subconsciously.

Furthermore, task decomposition can work in a hierarchical manner, breaking down the plan’s steps even further, offering scalability to even more challenging objectives [19].

Strategies for Generating Plans A variety of techniques can be selected when trying to achieve reasoning and planning with autonomous agents. They often include prompting the agent to generate the actual steps to explicitly induce the reasoning process and to observe how the agent proceeded with its solution. One popular approach is Chain-of-Thought (CoT) prompting [61]. Standard CoT involves instructing the LLM to output a single response containing the reasoning steps preceding the final answer (absence of an iterative workflow). This mechanism is then often used in combination with designing iterative agentic loops, such as the one proposed by ReAct [67]. So, the iterative nature comes from the control structure wrapped around the LLM, not the LLM utilizing CoT itself.

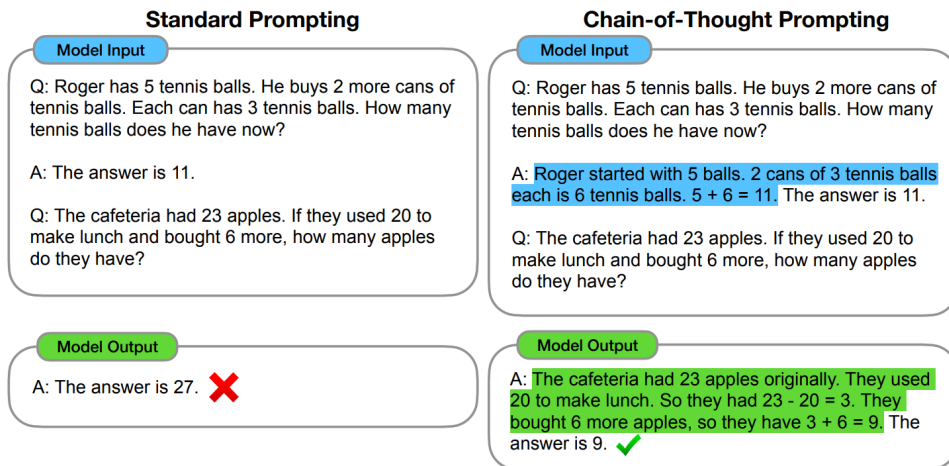


Figure 3.1: This figure displays the principle of Chain-of-Thought prompting. An example solution for a similar problem is included in the prompt to encourage the model’s reasoning. Figure taken from [61]

Generalizing over CoT, Tree of Thought (ToT) enhances the LLM’s problem-solving capabilities even further by allowing it to explore multiple reasoning paths and self-evaluate

the choices. This greatly increases the solution’s quality since the agent can learn from failed or unfeasible attempts and iteratively refine the approach [66]. ToT showcases that LLMs are able to thoroughly reason over many possibilities, generating coherent intermediate text sequences that can be used to backtrack or look ahead to make global choices.

Decomposition Types We can divide task decomposition methods into two categories: *Decomposition-First Methods* and *Interleaved Decomposition Methods* [27].

- **Decomposition-First Methods** Methods in this category process the original task and structure the objective into sub-problems first, and then plan for each sub-goal successively. In other words, all of the sub-objectives are identified first, then the whole structured plan is generated. The HuggingGPT agent [53] successfully employs this type of task decomposition.
- **Interleaved Decomposition Methods** Conversely, interleaved decomposition allows for alternating between decomposing the original objective and planning for sub-objectives. This means that the LLM can identify one or two sub-tasks at a time and directly plan for their completion. Afterwards, it continues decomposition with context enriched by its previous reasoning. CoT [61] and ReAct [67] both represent this category. While CoT embeds reasoning within the planning process, ReAct takes a step further by separating reasoning and planning into separate steps [27].

Both of these approaches have their benefits [27]. Decomposition-First Methods offer a close connection between the ultimate objective and the identified sub-tasks. This lessens the risk of deforming the solution by forgetting sub-objective details over time or hallucinating during reasoning. On the other hand, given that the sub-tasks are predefined, there must be an adjustment mechanism in place, because one unsuccessful step will fail the whole process.

Since Interleaved Decomposition Methods switch between decomposition and sub-task planning, they have room to dynamically refine the decomposition based on environmental feedback, ensuring that there is some tolerance for errors. However, as task complexity increases, hallucinations may occur because the model loses sight of the original goal.

3.2.3 Tool-Use

Inherent limitations of the powerful LLMs, namely, knowledge confined to training data, potential for factual deviations, inaccessibility to real-time data or external systems, and the struggle to perform precise and reliable calculations, are further addressed by allowing agents to use *tools* [49].

Tools can have various purposes, for example, connecting to external sources for information, executing actions, or handling complex math problems.

What Constitutes a Tool? A tool can be anything external to the model, available for the model to use anytime on its own volition (invoking the tool). Following is a list of possible types of tools that enhance the agent’s capabilities: search engines (e.g., Google API, Bing API); code interpreters (for executing outputted code); calculators; specialized APIs (e.g., weather, stock market); databases or knowledge bases; and even other language models (for example, a text-based model might invoke a multi-modal LLM as a tool if processing visual input is needed; this approach is seen in HuggingGPT [53]).

The code interpreter tool allows for defining custom tools programmatically and then teaching the model to output the correctly formatted tool call (e.g., calling a function to explore an index created from the user’s codebase). However, the code executed through a code interpreter tool usually runs in a limited environment to address security risks associated with exposing the LLM to the whole production environment and letting it „roam free“. These limitations usually include no network connection or limited access to the file system.

Core Mechanism Teaching the model when to use the tool, which tool, and how to invoke it correctly remains the biggest challenge in tool use [49]. Traditionally, prompt engineering methods were chosen to enlighten the model with the necessary information by including tool descriptions and examples of use in the prompt.

To enhance the accuracy and effectiveness of tool use, more advanced methods emerged involving fine-tuning LLMs to recognize specifically when a tool is needed and generate the corresponding API calls or function arguments (for the code interpreter). This represents a closer integration between the tools and the model itself [52].

ReAct [67] incorporates the tool usage inside the thinking loop (Thought – Action – Observation) as the Action step. The agent can autonomously decide to use a tool in the form of outputting the correctly formatted action call. The control structure detects this invocation by parsing the output and performs the corresponding operation. Once the tool’s results are available, they are incorporated back into the LLM’s reasoning process to further task progression. A diagram¹ depicting the combination of reasoning and acting is shown in Figure 3.2.

This addition opens completely new doors for the agent’s capabilities. Autonomous access to real-time information, the ability to perform complex operations, and the ability to interact with external modules or systems all contribute to the possible application of agents [49]. Specialized purposes can be achieved based on the specific requirements. These advancements allow agents to be more factually correct, reliable, and versatile.

3.2.4 Multi-Agent Collaboration

Although single agents powered by LLMs are already competent in solving (not only) software engineering tasks, more complex goals requiring multiple specializations and areas of knowledge benefit from the prospects of a collaborative environment. To handle these sophisticated challenges, Multi-Agent Systems (MAS) attempt to mimic the collaborative properties seen in human teams of developers [23, 58].

Core Concepts

Roles One of the fundamental ideas is to introduce different roles into designing the team of agents [58]. After decomposing a complex task into sub-objectives, they might each focus on separate areas of development. For example, a sub-goal of writing a few unit tests will require different reasoning paths for its completion than refactoring code

¹The diagram is a modified version of a schema from the Google Research Blog post ‘ReAct: Synergizing Reasoning and Acting in Language Models’ by Shuyu Yao and Yuan Cao, published November 8, 2022. Available at: <https://research.google/blog/react-synergizing-reasoning-and-acting-in-language-models/>

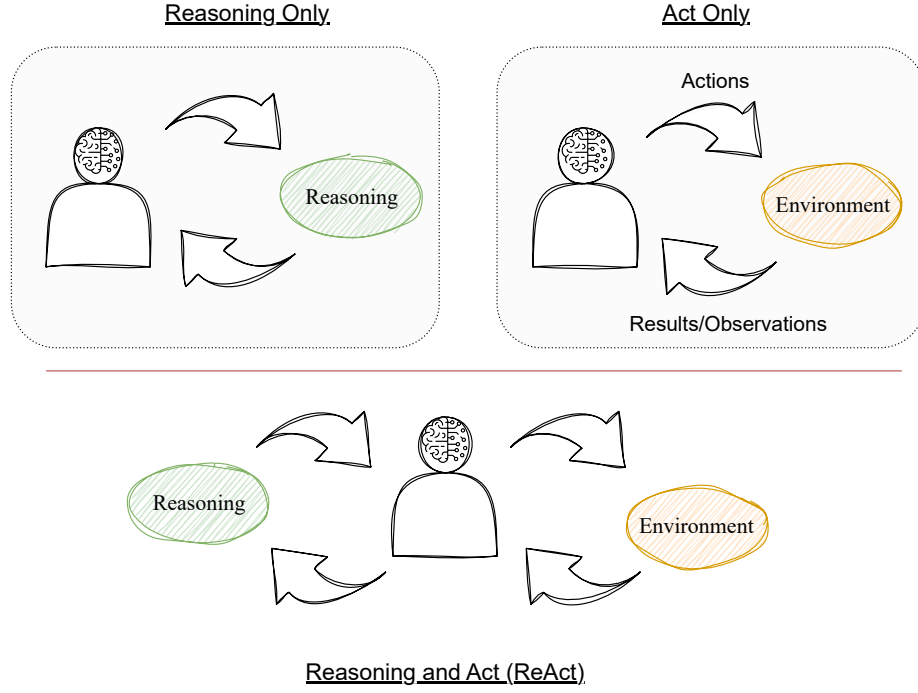


Figure 3.2: Schema of the core concept of ReAct. Agents are able to utilize both their reasoning capabilities and interaction with the environment in iterative loops.

to optimize performance. Of course, one LLM can have capabilities that satisfy multiple roles. In those cases, more agents can utilize the same model (separate instances of the same LLM) while incorporating different approaches to fine-tuning or distinct characteristics of their agentic control structures, such as the prompts being passed to the model.

This design facilitates division of labor and separation of concerns, ultimately enhancing the development process. Agents with clearly defined responsibilities are more focused and efficient in the system’s context.

Communication Communication mechanisms are a standard practice in MAS [62]. The agents need a way to, at the least, pass the results of their sub-tasks back to the central unit. However, this can be extended to them actively exchanging ideas using a „blackboard“ or a form of memory space to which the agents can subscribe [23].

Coordination The presence of a communication mechanism calls for inter-agent coordination. Specific strategies must be in place to manage the workflow, synchronize operations, and ensure that the team progressively works towards the shared goal [58].

A separate agent powered by LLM can be designed to govern the other agents [62], deciding which roles will be needed and distributing labor accordingly. Conversely, if a simpler functionality suffices, there might be only a static routing mechanism or a predefined workflow, such as the Standard Operating Procedures (SOPs) in MetaGPT [23], delivering the messages or directing the process as needed, without the enhancement of an LLM.

Multi-Agent Collaboration takes us a step closer to achieving a higher level of abstraction by representing a whole team of developers, capable of completing tasks involving multiple areas of expertise [58]. AutoGen [62] furthers this by offering an environment for quickly building new specialized agents to fit the needs of the specific application.

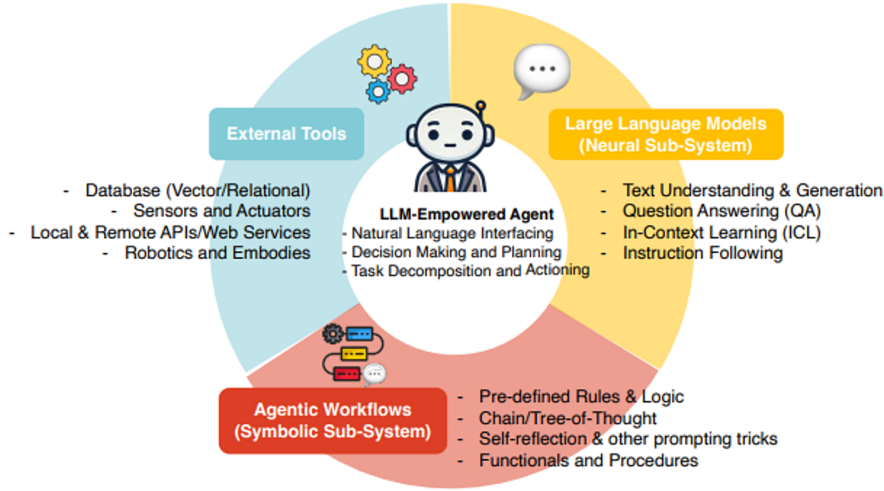


Figure 3.3: High-level architecture of an LLM empowered agent with its three main components. Figure is taken from [65].

3.3 Considerations

Although these advancements significantly increase the potential usage of these systems and the abstraction encapsulating the whole development process, there are consequences to the extra layers of logic.

- **Managing Overhead** Integrating the LLM into a control structure introduces more complexity into the overall system that needs to be managed [60, 63]. All agentic capabilities require additional code structures to implement them. The probability of an error occurring increases (caused either by a faulty output from the LLM or by the encapsulating layer itself) as the system grows. Therefore, all parts of the system need to be monitored and protected, for instance, with fallback values.
- **Hallucinations** Since we are trying to use the LLMs in these complex scenarios, we are deviating from its original design of single-turn interactions. Although beneficial, this puts a serious strain on the LLM’s context window and its implicit reasoning capabilities, possibly leading to hallucinations [26]. With very challenging tasks, the agent/agents can lose sight of the ultimate objective and shift their context in the wrong direction.
- **Selection of Approach** As we discussed, there are numerous ways to realize some of the agentic capabilities. Selecting the most appropriate implementation can be

challenging, especially in areas with nuanced requirements [60, 63]. Choices should be based on the specific use case, the workflow needed to facilitate the use case, and resource constraints. Experimenting with different options (e.g., model selection, task decomposition type, prompt engineering) can help to reveal the most suitable combination of approaches.

Chapter 4

Problem Specification and System Design

The software development process may be negatively affected by many factors [9, 42, 57]. This includes a loss of time due to an overwhelming amount of tasks that need to be done, while most are simple, repetitive assignments. In addition, unrevised code or a lack of code clarity further deforms software development. Satisfying the need for a streamlined, assisted process is an ongoing challenge that requires a great deal of automation and autonomous actions. In this chapter, we address the issues and define a system capable of mitigating them.

4.1 Problem specification

The main aim of this work is to streamline the software development process. This means saving the precious time of programmers by unburdening them of simple tasks that are easily automated, such as writing unit tests, documentation, or fixing bugs [18, 21, 68].

Developers frequently face scenarios where extending the implementation with new features requires a great deal of effort, often involving repetitive boilerplate tasks and detailed manual coding. Additionally, there may be issues where developers are allocating considerable amounts of time to understanding possibly unclear or messy code, either for maintenance, onboarding, or other purposes. Currently available tools typically operate at a lower abstraction level, demanding explicit coding instructions and deep engagement with the underlying code base structure. While some novel agentic systems, mostly oriented on building completely new projects, facilitate accepting abstract, natural language instructions and transforming them into structured assignments, a challenge remains in achieving abstraction in the development of local existing projects, where the developer is required to provide context of the established work. Therefore, there is a clear need for a system that can intake required functionalities through concise and abstract commands, but also autonomously acquires contextual understanding of the developer’s existing project. Interpretation of the high-level descriptions, obtained from the developer, is consequently contextually based within the referenced area [30].

4.1.1 Motivation

Streamlining the development process is essential for improving productivity, maintaining code quality, and reducing the time spent on repetitive tasks to a minimum. When devel-

opers become overwhelmed by routine tasks, such as documentation, functionality tests, or bug revision, they have significantly less time to focus on the more valued, high-level aspects of the process, including system architecture, feature innovation, or optimization. Automating repetitive tasks that strain human resources brings the prospect of saving costs and time, improving consistency, and minimizing human error [36, 47]. Subsequently, it allows programmers to explore more complex matters that automated systems would not easily complete.

In real-life scenarios, development teams might often find themselves in situations where keeping to the timeline becomes extremely difficult due to accumulated minor issues or poor code sustainability. For example, a team working on a large-scale business application might encounter a delay in their latest release cycle, not caused by complex new integrations but by an overflow of tedious yet necessary code refactoring, extensive testing, or updating outdated documentation.

An enhanced, assisted approach would proactively resolve these issues by completing routine tasks as quickly as possible autonomously and enable the team to reserve effort for complex tasks, directly impacting project quality, system innovation, and delivering the product on time.

To examine the positive effects of using automation tools built on large language models, a controlled experiment with GitHub Copilot was carried out by [47]. Their findings are shown in Table 4.1. The experiment revealed that the group using Copilot completed their programming tasks approximately 55% faster compared to the control group. This demonstrates that automation based on large language models significantly enhances developer productivity, especially assisting less experienced programmers.

Table 4.1: Heterogeneous Treatment Effects of GitHub Copilot Usage [47]

	Estimates	SE	t-Stat	p-Value
(Intercept)	78.01	67.84	1.15	0.2552
Programming experience (years)	8.23	4.36	1.90	0.0629
Hours of programming per day	-11.70	4.74	-2.47	0.0168
Age: 25–44	-74.55	33.52	-2.22	0.0303
Unemployed	-35.98	36.33	-0.99	0.3263
Income < \$20,000	0.64	27.47	0.02	0.9814
No college	-36.57	32.89	-1.11	0.2711
Language Preference: Java	-11.77	33.16	-0.35	0.7240
Language Preference: Python	22.90	42.19	0.54	0.5895

Note: Adapted from Peng et al. [47]. This table presents heterogeneous treatment effects. Developers with fewer daily programming hours and those above age 25 were significantly more likely to benefit from Copilot, while high-frequency coders saw smaller productivity gains.

4.1.2 Acknowledging Progress

The various shortcomings of the development process and the possibility for improvements initiated the emergence of autonomous AI assistant systems [60]. They significantly reduce the developer’s workload and perform various actions with an understanding of the existing codebase [47].

The MetaGPT and AutoGen frameworks showcase the potential of multi-agent approaches, inspired by human developer teams, where agents with specialized roles (e.g.,

planning, coding, documenting) interact and coordinate to reach a shared goal. Implementations like LocAgent [12], a tool utilizing an LLM agent for code localization guided by code graphs, ensure that agents can process even highly complex repositories, allowing for high scalability. Agentic patterns offered by foundational works, such as ReAct [67], enable synergy between reasoning and action, allowing agents to interact with tools in the action step. The tool’s outputs subsequently aid the dynamic planning of the next steps. Additionally, even commercial systems, for example, Github Copilot or Amazon Q Developer, are shifting towards agentic features, scaling their application to more complex problems, not limited to multi-file code edits or guided refactoring based on natural language prompts.

Furthermore, as was mentioned before, AutoGen [62] enables users to build new agents in a prepared environment, emphasizing the easy and quick design and the adaptability of the system to nuanced requirements. The LangChain framework¹ is another example that allows its users to develop LLM-based applications using various predefined agent types. This approach can pave the way for future ambitions of making AI development more accessible by allowing users with minimal coding knowledge to design new agents through a user-friendly interface.

4.1.3 Current Limitations

While offering many advantages of streamlining routine software development, the current frameworks and tools are facing various challenges and limitations as well.

- **Limited abstraction and creativity:** Despite their planning and reasoning capabilities, current agentic AI systems may still lack the necessary genuine understanding, primarily pertaining to how the agents can access and process existing code. The agent’s control structure is designed to guide it to follow intricate instructions and generate detailed plans based on patterns, but tasks that require autonomously obtaining and understanding implicit or referenced information (from local codebases) may be out of reach for these systems [28].
- **Shallow contextual awareness:** While basic LLMs struggle with local context, agentic systems encounter a magnified issue. For the agents to plan and perform complex operations, such as refactoring across multiple files or writing new code impacting numerous modules, they often need a deep and accurate understanding of the entire project, its dependencies, and domain logic. Processing the whole complex project by a single LLM’s context window is not feasible in the current state of LLMs, leading agents without context enhancement mechanisms to make incorrect assumptions or break distant dependencies [31]. This challenge can also appear in documentation generation. AI tools are able to effectively describe *what* a piece of code does but not *why* it exists or how it integrates into the rest of the project, meaning important dependencies, necessary for the developer’s understanding, are omitted from the documentation altogether. Indexing the existing codebase and creating graph representations for code retrieval helps mitigate this issue, but preserving accurate and detailed context remains difficult.
- **Hallucination in Planning and Execution:** In agentic systems, hallucination has a greater impact than just incorrect code snippets. Since agents perform numerous intermediate steps, deviation from the expected flow is more likely to occur throughout

¹LangChain’s open – source project can be found at <https://github.com/langchain-ai/langchain>

this process. Agents may exhibit hallucinated plans, reasoning steps, or non-existent tool invocations. For example, an agent utilizing the novel ReAct [67] framework might hallucinate the outcome of an API call, sending it down an incorrect reasoning path and degrading the subsequent steps. This makes it more difficult to debug the complex workflow of agents since hallucinations can occur at any point in the task solution process [26].

- **Robustness, Reliability, and Error Handling:** Given that the agents often interact with external factors, such as APIs, file systems, specialized software, or other tools, they face issues, not only caused by the foundational LLM. An agent might receive an unexpected API response, encounter file permission issues, or bugs in its own control structure, causing it to fail. AutoGen [62] provides means to define custom agent interaction, though maintaining workflow resilience to errors in the different steps of the process is an ongoing challenge [25]. Agents should handle these issues gracefully so that, if possible, an error in one step does not cause the whole agent to fail.
- **Limited adaptation to project conventions:** Keeping unified coding practices, standards, and conventions is essential for the stability of software development [6]. Agentic assistant systems performing modifications significantly impacting the existing codebase (e.g., new features, multi-file refactoring) must conform to these project-specific conventions. Even though the conventions and constraints can sometimes be deduced from the codebase itself [6], the issues with the agent’s context window length prevent it from accurately identifying and maintaining them throughout complex tasks [64], forcing developers to define them explicitly in the assistant’s prompt. Even then, the agent can still forget these instructions, leading to a great deal of manual correction.
- **Computational Cost and Latency:** LLMs themselves present challenges of high computational requirements and varying operational latency. Since agentic approaches effectively wrap an LLM in extra layers of logic, these challenges amplify. Agentic systems often rely on complex, iterative workflows that require multiple interactions with the base LLM. Each of these steps introduces some amount of latency based on the specific model. This means that the overall latency of the system is determined by the collective delay of all the LLM calls performed during the workflow [60, 2]. Furthermore, an agent interacting with external systems as tools inherits their latency as well, since it needs to wait for their results before proceeding with task completion. Computational costs also increase considerably with larger agentic implementations. As the task complexity rises, more detailed decomposition is needed, resulting in more sub-tasks, which leads to more model inference. Consequently, energy consumption is strained, raising environmental considerations [2].

These challenges represent ongoing issues that are under active research. Creating reliable and robust agentic systems requires careful selection of the most suitable approaches and novel architecture design.

This thesis proposes a multi-agent assistant system aimed at the automation of routine software development tasks with a high level of abstraction of providing relevant information and a focus on enhancing codebase context through an integrated search and retrieval tool.

4.2 System Design

To enhance the overall process of software development, this thesis proposes a modular multi-agent system leveraging the capabilities of large language models. It was designed to enable the autonomous completion of routine coding tasks, such as documentation, testing, or bug fixing, based on high-level, natural language requests from the user.

The system comprises various specialized agents that collaborate through a shared memory structure coordinated by a designated communication leader. Individual agents are responsible for automating a separate part of the software development process. This architectural approach effectively accommodates scalability, agent independence, and the possibility of seamlessly integrating new functionalities.

This section will go over the overall structure of the system, the roles of individual components, and the communication mechanism, and outline the conceptual flow of completing tasks. The design process was fueled by the need for clarity, abstraction, and adaptability to numerous codebases and programming tasks.

4.2.1 System Process Overview

The following outlines the operational flow of the proposed multi-agent system. It accepts an abstract natural language query from the user and either provides a detailed answer to the question or decomposes the assigned task into executable actions and delegates them to the specialized agents. Iterative loops are included in the flow to allow agents to gain more contextual awareness and refine their thought processes, further enhancing the quality of the output.

Natural Language Input The system begins its operation process when a developer formulates a request in natural language, for instance, “Generate documentation for the file `person.py`”. The query is accepted by the system and passed down for further processing.

Mode Determinator A dedicated model is prompted to determine which of the two operational modes should be selected: **clarification** or **completion**, based on the assigned query. In clarification mode, the *Analyst Agent* explores the local codebase in order to understand the referenced segments and generates helpful explanations that provide meaningful insight to the developer. In the case of completion mode, the system continues with intent classification.

Intent Classification The query is first processed by the *intent classifier*, which consists of a fine-tuned BERT-based model to extract predefined intent classes from the query [43] (e.g., `write_tests`, `optimize_performance`). This informs the downstream components about which agents will be needed to complete the task.

Codebase Analysis and Task Planning Once intents are established, the query and corresponding codebase are passed to the Analyst Agent. This agent is augmented with an indexing tool that is able to generate a graph-based representation of the provided codebase [12]. To determine segments of the project that are most relevant to the assigned task, the generated index is used to perform semantic searches over the codebase [30]. The analyst agent then uses this acquired context to produce one or more structured, code-specific sub-tasks tailored to the specific objective. If sub-task dependencies are present,

the Analyst Agent marks them accordingly, signaling the specialized agents to wait before the prerequisite tasks are completed.

Task Distribution via Shared Memory The Analyst Agent inserts the generated sub-tasks into a shared memory buffer managed by a central *Communication Leader*. The leader component oversees the shared memory and waits for incoming task assignments in the form of messages. Once obtained, it routes each task to its corresponding *target agent*, such as the Test Agent or the Documentation Agent, using task queues assigned to individual agents. This centralized messaging pattern is noted to improve efficiency [20] and resembles the shared message pool proposed by Hong et al. [23].

Task Execution by Specialized Agents Upon receiving a task, the specialized agent utilizes its large language model to begin completing the task. The agents work iteratively – continuously examining the code context and reflecting on the achieved functionality to refine their answers [67]. For example, the Test Agent may generate a suite of unit tests based on the file context and the original user query. The outputted results are then transferred back to the shared memory in the form of result messages.

Aggregation and Final Output The Communication Leader collects all the results and forwards them to the Analyst Agent, who reviews the received answers and constructs an appropriate message to the developer.

This structured pipeline ensures modular composition, parallel task processing, and the possibility of extending the system via new agents without significantly impairing the existing system components.

The full interaction flow is depicted in Figure 4.1.

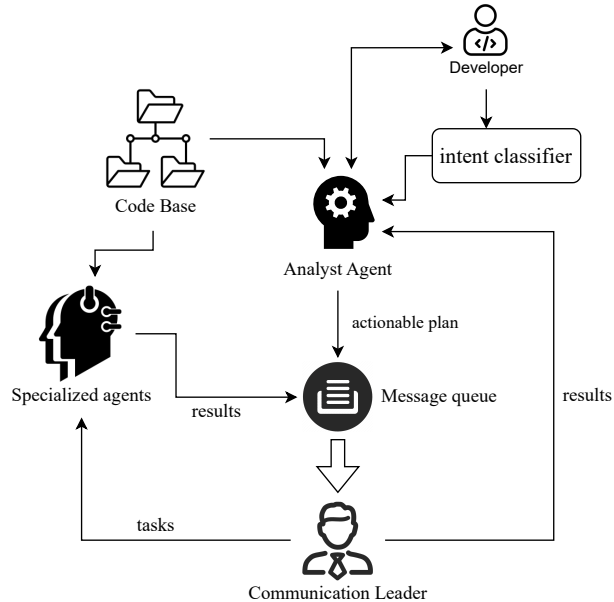


Figure 4.1: Architecture overview of the multi-agent system and its message flow.

4.2.2 System Components

Each component has predefined responsibilities and plays a vital role in maintaining the overall functionality of the system and enabling the necessary abstract workflow described above.

Intent Classifier This component was motivated by the benefits of easily identifying required resources for the task at hand [43]. Predefined intent classes were chosen based on the most common coding activities – `debug_code`, `add_feature`, `write_tests`, `document_code`. A chosen BERT-based model analyses the original query from the developer and assigns probability scores to all of the intent classes. For example, a query „Test the functionality in `person.py` and generate a docstring for each function“ will most likely receive scores close to 1 for classes `write_tests` and `document_code`.

Analyst Agent The Analyst Agent embodies two roles. In its clarification mode, it is responsible for analysing the developer’s questions about the local codebase by searching for the explicitly and implicitly mentioned areas and providing detailed explanations, possibly enriched with relevant code snippets. In its completion mode, it serves as a team leader of sorts, decomposing the larger objective into smaller, more manageable tasks and assigning them to individual specialized agents [16]. It reasons over what needs to be done and what dependencies might be affected by the necessary changes.

The Analyst Agent’s completion module is composed of two mutually cooperating segments: a planning component responsible for generating concise sub-tasks for the specialized agents and an external, open-source tool, LocAgent [12]. This tool, powered by its own large language model, is capable of indexing the developer’s existing code and subsequently performing semantic searches over it to identify the most relevant segments [30], which are then inserted into the planning module’s thought process.

While not developed as part of this system, LocAgent has been integrated into the Analyst Agent to assist it in making informed, context-aware decisions [30].

Specialized Agents Specialized Agents form a team of four capable units, each responsible for a separate type of assignment corresponding to the four predefined intent classes: *Implementation Agent* – responsible for adding new features; *Debugging Agent* – localizes and resolves coding errors; *Testing Agent* – writes test cases to assert functionality; *Documentation Agent* – generates documentation to ensure clarity.

All agents actively wait for tasks to appear in their assigned queues. Once retrieved from the queue, the agent begins its iterative loops of operation. A simple iterator was designed for this purpose. An agent iterator is a mechanism that allows the agent to generate its final output over multiple steps instead of demanding an answer on the first try [54]. One loop of the iterator generally has a structure that might resemble this flow: **Thought** – **Action** – **Observation** [67], where the **Action** represents the agent’s request for more context and **Observation** represents its reasoning over the acquired information. This form of self-guided iteration has been shown to improve performance in reasoning-intensive tasks [67].

Once the agent is confident that it has reached a satisfactory solution, it outputs the final answer and sends a result message to the shared memory component.

Communication Leader The Communication Leader is not an agent powered by a large language model, but rather a separate process ensuring smooth message processing and task assignment. When messages appear in the shared memory component, it alerts the leader, which processes all of them and subsequently inserts tasks into the agents' queues [23].

Shared Memory This resource is represented by a shared data structure, stored in the memory of the program. Structured messages are inserted into the structure and retrieved by the Communication Leader for processing.

A network-based messaging mechanism using packets was explored as well, but this introduced latency and the risk of packet loss during transmission [62].

To manage sub-task dependencies, this component incorporates a mechanism for tracking the completion status of tasks. This is handled internally by a dedicated structure, holding the status, results, and identified dependencies for each sub-task. Specialized agents query this structure via Shared Memory to make sure the prerequisites are met, and if not, they pause execution until the dependencies are satisfied.

These components collectively form a reliable and flexible system, suitable for automating routine coding tasks. Through the division of intent classification, code analysis, planning, task execution, and coordination, the system preserves clarity and separation of concerns. Integrated "index and search" tool enhances the system with contextual information about the user's project [30]. All of this is essential for transforming the abstract query into accurate solutions.

Chapter 5

Implementation Details

This chapter will focus on explaining the technical details behind the realization of the multi-agent system proposed in the previous chapter, called CogniCollab.

Implemented primarily in Python, the system leverages novel libraries, such as `litellm` and `openai` for LLM inference through external APIs, complemented by libraries like `threading` and `queue` to handle the intricate interactions and control logic were utilized to implement the intricate interactions and control logic.

The modular approach, involving distinct agent components, a shared communication layer, and a static coordinator, boosts separation of concerns. Individual agent roles are defined based on careful prompt engineering, encouraging the seamless integration of new roles. CogniCollab was implemented to work with local Python projects, but can be very easily adapted for other programming languages as well.

5.1 Environment and External Tools

Core aspects of the system’s implementation include its development environment, setup steps, and necessary technologies. These properties are detailed below, followed by the explanation of how LocAgent, an external tool, was integrated with the environment to enhance the agents’ contextual capabilities.

5.1.1 Environment setup

Python (specifically version 3.12) was selected as the primary programming language for this work. The system runs in a dedicated virtual environment created through Miniconda. This approach is explicitly requested by the LocAgent’s setup instructions to accommodate complex dependencies and ensure consistency. Therefore, a Conda environment was used to house all the packages specified in `src/LocAgent/requirements.txt`.

Numerous crucial libraries ensure the overall system functionality. While `openai` enables connection to OpenAI-compatible API endpoints authenticated with an API key, `litellm` serves as an abstraction layer, encapsulating and simplifying the calls to various LLMs APIs, handling model specifics, and incorporating retries. Together, they ensure the smooth interaction with LLMs. Concurrent behavior for multi-agent collaboration is managed by the `threading` and `queue` libraries. Agents, operating as separate threads, employ a message queue for incoming tasks, forwarded by the Communication Leader.

A unified format of messages is ensured by the `json` library, enhancing data exchange. To instantiate the external LocAgent tool as a separate process (specified by LocAgent’s

documentation), the **subprocess** library is utilized. Lastly, **re** is leveraged for regular expression matching, namely in extracting the generated plan from the Analyst Agent and identifying actions in reasoning loops.

5.1.2 LocAgent Integration and Adaptation

To address the inherent limitations of the context window length of LLMs, this system integrates and adapts the external tool LocAgent [12]. LocAgent was originally designed to search for and identify relevant segments of code, from specified GitHub repositories, based on natural language queries. By creating a graph-based representation of the available codebase, it enables its underlying LLM to perform complex reasoning and accurately identify relevant sections of code even if they are omitted from the query.

Augmenting this tool with the Analyst Agent is essential for providing the crucial context for quality plan generation. Furthermore, each agent is provisioned with the LocAgent’s search & retrieve capabilities over graph-based representations.

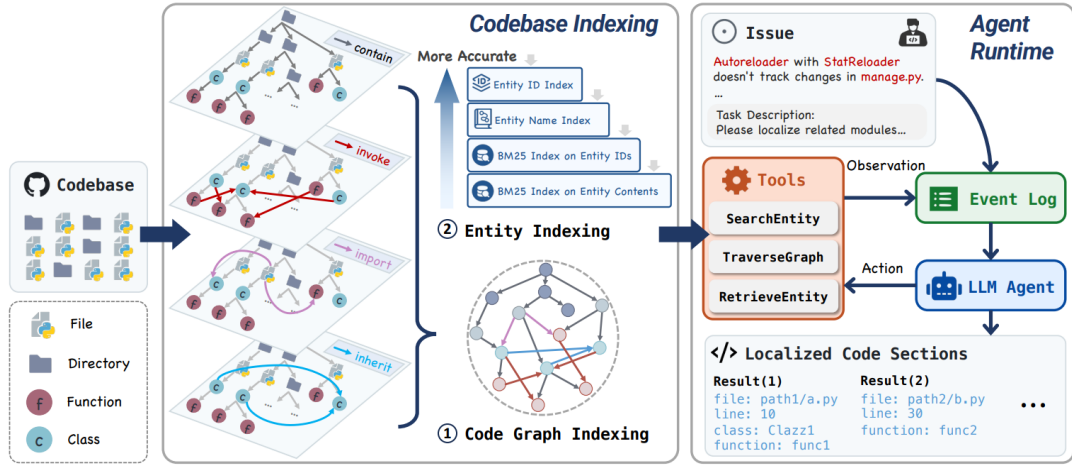


Figure 5.1: Overview of the LocAgent framework. The codebase is first parsed to create a graph-based representation. Then, LocAgent constructs sparse indexes that allow exploring the project structures and searching code context. Figure taken from [12].

Motivation Leveraging LocAgent goes beyond simple keyword matching. It offers a deep understanding of the code’s context, allowing unlimited access to the codebase. At the same time, the system guides the embedded LLM in generating explicit reasoning about the relevance of the analysed code.

The Analyst Agent (AA) benefits from the detailed contextual insight acquired from LocAgent. The relevant information is integrated directly into the AA’s reasoning.

Integration Mechanism AA utilizes the full functionality of the LocAgent tool – the *PlanningModule* component creates a subprocess (using the `subprocess.run` method of the *subprocess* library) through a terminal command that runs the LocAgent’s main script (`auto_search_main`). Executing the main file from the command line is in accordance with the LocAgent’s GitHub documentation.

The tool’s initialization call parameters provide further information to the system. The user query is passed through `-query`, `-model` is reserved for the LLM selection for codebase analysis, `-instance_id` uniquely identifies the specific issue, and `-repo` passes the local repository path (this parameter was added during the integration to allow processing of local projects instead of remote GitHub codebases).

LocAgent’s Role The tool’s full functionality is triggered before each decomposition and planning phase. During its operation, a graph-based representation of the codebase is constructed and stored in a separate file for future access. The LocAgent’s LLM analyses the code while focusing on the received query.

The result of the tool, consisting of identified relevant files, classes, and functions (including code snippets and short summaries), is passed to AA before the planning phase is initiated and incorporated into its prompt, explicitly marked as the output of codebase analysis. This significantly reduces the AA’s otherwise broad area of focus to nudge the agent in the right direction. Integration with LocAgent represents a form of Retrieval Augmented Generation (RAG) [55], where the prompt presented to the LLM is enriched with retrieved helpful documents or other content.

Additionally, components of the LocAgent system aid individual agents further by the utilization of the prepared graph-based index to retrieve results of code searches using specialized LocAgent’s functions (leveraged also by the tool’s underlying LLM). The agents’ intermediate outputs are parsed by the LocAgent’s ResponseParser to successfully detect requests for search and the tag marking agent’s final response using regular expressions.

All of the agents also re-use a Jinja2 prompt template, `system_prompt.j2`, implemented in LocAgent, that provides them with the necessary descriptions of specialized code search functions and their parameters.

Data Exchange Since the tool runs as a sub-process, the result of the analysis needs to be reliably transferred to the main execution. In general, when retrieving a result from a sub-process, a practical solution is to let the process output its final answer to the standard output pipeline and then capture it in the main process. However, since LocAgent was designed to be able to handle multiple complex issues, it logs various information and outputs intermediate messages from the underlying LLM’s conversation, which overloads the standard output and prevents from easily isolating the necessary information. Instead, LocAgent writes the findings into a temporary file, stored in a folder specified during the tool’s call. Subsequently, the main process can read this file to extract the structured localization result.

Adaptions Made The tool was adapted to handle local repositories instead of remote GitHub projects, and the prompt guiding the LLM in its reasoning steps was modified to accommodate the nuanced needs of the system proposed by this work.

New Jinja2 user prompt templates were created for the individual agents and stored in the folder handling prompt logic in the LocAgent system. These templates show an example workflow of the task type they are required to solve, by giving them example reasoning steps and search invocations.

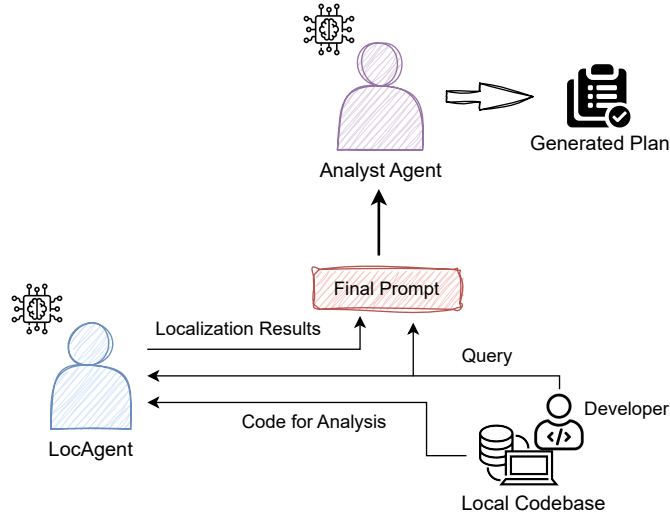


Figure 5.2: CogniCollab employs principles of RAG through the integrated LocAgent, which retrieves necessary information from the specified codebase.

5.2 LLM Interaction

In multi-agent systems, it is beneficial to abstract the model interactions with a unified interface. There is a clear need for a consistent way of handling the API calls, managing model configurations, and instantiating agents with different, specialized prompts.

To facilitate this concept, CogniCollab employs the class `UnifiedAgent`, defined in `unified_agent_class.py`, which encapsulates the initialization of API clients and basic model operations.

Initialization The class holds the necessary information for executing the API calls. It is initialized with values, specifying the selected model's name and the API key to be used for authentication. An OpenAI API client is constructed from the API endpoint of the model's provider and API key using the `openai` library. This client is stored in an instance attribute for usage in all API calls. Furthermore, LocAgent's `PromptManager` class is leveraged to load and render the Jinja2 system and specified user prompt (based on the agent currently initializing) templates.

Inference For an actual interaction with the agent's model, the class incorporates a `generate_completion` method. This method receives the current state of the agent's internal conversation (a list of messages structured as JSON objects) and utilizes the `litellm.completion` method of the `litellm` library, which simplifies the API calls. Passing the conversation state with the call effectively allows the model to continue its reasoning from where it left off. The completion method's parameters include the model name to use the correct LLM associated with the specific agent.

This approach of sharing a unified class encourages **code reuse**, simplifies **maintenance** by centralizing LLM interaction and logic, allowing easy functionality changes, and strengthens **modularity**, improving the overall development of the system.

5.3 System Orchestration

To make the assistant tool usable, separate components need to be orchestrated together in a clean workflow. For a comfortable interaction with this system and a concise structure, a simple Python application using the **Streamlit** library was built.

This upper layer provides means to input queries, select codebases stored locally, and maintain chat history, all within a user-friendly interface. The application facilitates the initialization of the main agent components and manages their lifecycle throughout the user’s request, from the initial prompt to the result.

User Interface and Input Handling The developer can formulate their task assignment or question into a dedicated text area. Streamlit does not support native folder picking, so to facilitate manually selecting a local project through a separate pop-up, this application utilizes a sub-process to handle this operation. Each time the corresponding folder selection button is pressed, the script `lib/tk_dir_selector.py` (this script utilizes the `filedialog.askdirectory` method from the **tkinter** library) is triggered via `subprocess.Popen` to open a directory selection dialog. The choice is subsequently captured from the sub-process and propagated to the user interface (UI).

Once the query is written and a local folder is selected, the developer can initiate the workflow using the **Process Query** button. The application signals processing and displays the result when finished. The results are also stored in a history of messages.

The application is complemented by a helpful description of the tool and how to interact with it.

To maintain the UI state across refreshes, the application script utilizes `session_state` from the Streamlit library, capable of storing various values (e.g., current query, selected folder path, result history).

Main Execution Once a new request is sent to the system by pressing the Process Query button, the execution process is first prepared: the session state’s **processing** flag is set, a queue for the result is created, and a background thread that handles the initialization of the individual system components is started. This thread calls the `set_current_issue` function from the LocAgent system, which sets the global variables for the specific selected local project (creating/retrieving the graph-based codebase representation). The necessary classes are instantiated while carefully configuring the inter-class references (e.g., passing the agent queues to the coordinator’s initialization). Subsequently, the agent threads and the coordinator thread are started and inserted into `session_state.background_threads` for background processing.

Once the coordinator detects that all sub-tasks have been completed by the agents, it triggers the AA’s `consolidate_results` method. The final answer is then inserted into the dedicated result queue created by the main Streamlit application. During processing, the application waits for the result by periodically polling the queue without freezing the UI. After the result is acquired, it is presented to the user, and the chat history is updated.

5.4 Analyst Agent

The Analyst Agent serves as a front desk team leader, accepting the developer’s requests, analysing and translating them into manageable sub-goals, including objectives not explicitly stated by the user, or a detailed explanation. This versatile functionality is achieved through a combination of internal agent logic and specialized component modules, triggered based on the nature of the incoming request.

AA consists of two main modes, underlining its role and purpose in the system: *Clarification* mode and *Completion* mode (incorporating the planning phase).

Agent Overview This central agent is represented by the `AnalystAgent` class defined in `analyst_agent.py`. It is instantiated with the API key for inference authentication and a reference to the shared memory structure, which was instantiated beforehand.

During initialization, individual sub-modules for the clarification and completion modes are created and assigned to AA. Additionally, the agent is augmented with an LLM-based mode identifier that analyses the developer’s query and categorizes it into one of the two modes.

The main logic of this agent resides in the `process_task` method, which receives the assigned task, structured as JSON, and an instance ID, uniquely identifying the current issue (this value is propagated to downstream sub-modules and will be used for storing the generated graph-based index for repeated access). The problem statement is passed to the mode detector, represented by the `ModeClassifier` class defined in `mode_determination.py` in the `analyst_agent` folder, for classification, after which the corresponding AA’s sub-module is triggered to begin task completion.

The `process_task` method returns the result of the chosen sub-module. In the case of the completion mode, it also extracts the generated plan from the planning component’s response and sends a `subtasks_init` message to the shared memory structure, which internally sets up the sub-task dependency management and distributes the assignments via separate shared memory messages.

5.4.1 Mode Determination

Since this component is responsible for a straightforward two-way query classification, no complex logic is needed for its implementation. The `ModeClassifier` class includes the vital `detect_mode` method, in which the underlying LLM is simply instructed to classify the following software development query into „completion“ or „clarification“ categories, with short descriptions of both included in the prompt.

Subsequently, the detector’s model outputs the corresponding word, either clarification or completion. This result is then used in the main `AnalystAgent` class.

Accurately identifying the type of requests made by the developer is necessary for ensuring the smooth functionality of both AA’s modes.

5.4.2 Clarification Mode

This mode, embodied by the `ClarificationModule` class, is responsible for clarifying code objects and logic to address challenges of low code clarity and aid onboarding in real-world development.

`ClarificationModule` reuses the unified LLM abstraction class, `UnifiedAgent`, to encapsulate its embedded model. To avoid the overhead of executing the full functionality of

the LocAgent system for this single-turn type of interaction (question – analysis & reasoning – explanation), this module leverages only the reused `agent_logic_loop`, which guides iterative reasoning with a specified problem statement and agent prompt. While this approach omits the detailed analysis summary from LocAgent, it still allows the embedded model of this module to perform its own exploration (using the LocAgent’s graph index and specialized code search functions). The underlying model, guided by the reasoning loop, is capable of tracing any detected dependencies with additional searches to enhance code understanding for quality explanations.

5.4.3 Completion Mode

The Completion Mode incorporates the `PlanningModule` class, responsible for codebase analysis (through the LocAgent tool), task decomposition and plan generation, and constructing a response based on the results from individual agents.

`PlanningModule` first leverages the LocAgent system by executing the tool’s main script as a subprocess and providing the necessary issue information for its operation. After the analysis completes, the results, together with a complete list of available files in the codebase for completeness, are embedded into the agent’s prompt. Even though the agent is provided with the detailed analysis, it is encouraged by the prompt to perform any additional searches if more context is needed for nuanced decomposition. The `PlanningModule`’s underlying LLM, encapsulated by `UnifiedAgent`, enters the `agent_logic_loop` to iteratively reason over the presented information and generate an actionable plan based on the available agents. The final plan is passed to upstream logic that handles task routing via the shared memory component.

5.5 Intent Classification

This process represents the first step within AA’s completion mode. It serves to provide context about the type of task (or tasks) the developer assigned to the system, based on analysing the natural language query.

Accurately identifying intent supplies the subsequent plan generation with additional guidance by hinting at which agents will be needed to complete the task. The method ensures that the decomposed sub-goals are relevant and appropriately assigned to the individual specialized agents. This way, AA is focused on the intended operations from the beginning.

To implement intent classification for CogniCollab, a pre-trained, code-aware language model was fine-tuned for multi-label text classification, as an alternative to relying on simpler methods (for example, keyword matching) or inference through external APIs, to showcase the importance of fine-tuning in AI assistant system development.

5.5.1 Model Selection and Dataset Preparation

For this task, a large language model prominent in deep language understanding and text classification is needed to correctly map predefined intent classes to the developer’s query. A general-purpose model trained on general English text, `bert-base-uncased`, was initially considered for this module, but the CodeBERT model represents a more suitable candidate.

While both utilize the novel BERT architecture, CodeBERT was pre-trained on a massive dataset of paired code snippets and their corresponding natural language descriptions.

This enables the model to learn the semantic relationship between the natural language phrases and the corresponding programming concepts, meaning it can better interpret coding-related questions and assignments.

Dataset Preparation In the pursuit of suitable data for code-related intent classification, publicly available datasets were explored. However, typical intent classification datasets are oriented toward general conversational or business-oriented intent classes, not really appropriate for this system.

GitHub commit messages proved to be a plentiful source of the needed data. By searching for repositories, using code-related queries, many commit messages can be extracted for further processing. The downloaded messages were filtered to omit entries that are too long/short or include inappropriate sequences. Data labeling was carried out programmatically, using a keyword-based mapping, followed by a manual review. The predefined intent classes were selected as follows: `debug_code`, `add_feature`, `write_tests`, and `document_code`.

Analysis of the dataset revealed that the `debug_code` intent class was significantly overrepresented, potentially introducing a bias towards that intent. To mitigate this issue and improve performance across all intents equally, an **upsampling** strategy was utilized. The core concept of this strategy is data augmentation via synonym replacement. Using this approach, samples with underrepresented intent classes are identified and their key terms are replaced appropriately with synonyms to create similar yet new samples. Crucially, the original intent labels were preserved for these synthetically generated samples. This process naturally increases the number of examples, leading to an expanded and more balanced dataset. The resulting dataset contained around 1200 examples. While this is a relatively small size for a dataset, the use of the pre-trained CodeBERT model ensures that it can be sufficiently effective to reach prominent intent classification capabilities.

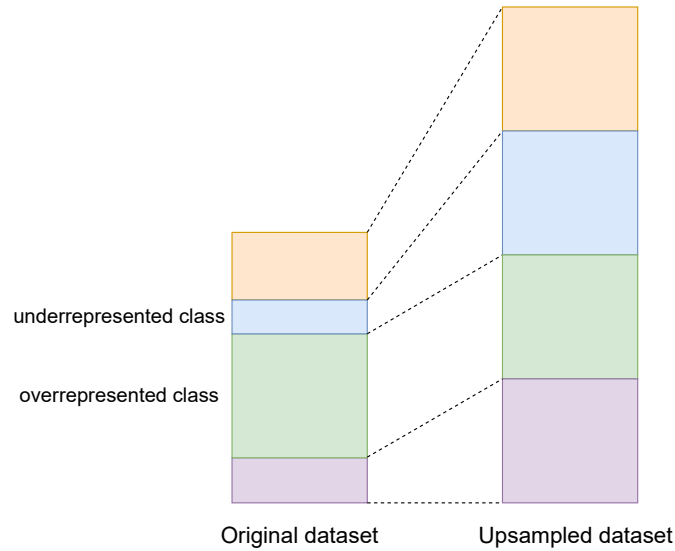


Figure 5.3: Depiction of the upsampling strategy, which balances out the dataset across classes by augmenting underrepresented ones.

5.5.2 Fine-Tuning

After preparing the dataset, it was used to fine-tune the CodeBERT model, freely accessible on the HuggingFace platform. Key libraries, used for this process, include **transformers** (providing means for loading models and tokenizers and supplying the Trainer class for facilitating fine-tuning) and **datasets** (for handling the data). The task formulation was framed as a `multi_label_classification` problem.

Before the fine-tuning process begins, the selected model is first loaded using the `AutoModelForSequenceClassification` class from the transformers library. The data is tokenized, and the LLM enters the main fine-tuning process with carefully chosen training arguments. This configuration significantly impacts the training’s outcome. During training, the system monitors the F1 evaluation metric for best model selection. After the process completes, the model is stored locally in a dedicated folder to be later used by AA.

5.6 Agent Execution

The core of agent operation lies in the `agent_logic_loop` control structure defined in `agent_logic.py`. It offers a standardized iterative execution mechanism shared among all agent types. This loop facilitates multi-step reasoning and tool usage and allows the agents to observe intermediate tool results to refine their approach.

The agent’s LLM (wrapped in `UnifiedAgent`), problem statement, and specialized prompt are passed to the function to configure the operation for a specific reasoning loop.

Preparation The agent’s context of the thinking process is maintained across steps (multiple LLM calls) through the `messages` list of message objects. This structure holds all of the previous messages, including the initial prompts. The agent’s `system_prompt`, `user_prompt`, and specified specialized agent prompt are inserted into the list first, followed by the developer’s issue description.

Main Execution Loop After the initial conversation state is prepared, the agent enters the main iterative behavior. Here, the agent outputs reasoning steps and action calls to progressively complete its task.

LLM is prompted using the `generate_completion` method of the `UnifiedAgent` class. The messages list appends the model’s response to update the conversation state in each iteration. This function utilizes the `ResponseParser` class of the LocAgent system, which parses the LLM output and detects present actions.

Identified `run_ipython` actions are passed to the `execute_ipython` function of the LocAgent system, which executes the generated call, performing the corresponding code search. The tool’s result is appended to the list of messages as an observation.

Once the agent is confident in its solution, it outputs the `finish` action, which terminates the execution loops and considers the message containing the finish mark as the agent’s final response. Alternatively, there might be an issue with the underlying LLM causing the agent to struggle with completing the task, in which case, there is an iteration count limit to prevent endless execution.

5.7 Communication Infrastructure

This section details the individual system components, which ensure asynchronous communication and coordination between the concurrent agent threads. Given that the agents work independently yet operate on the same codebase, a robust method is needed to distribute tasks from AA to the specialized agents and maintain code consistency with subsequent objectives.

The solution comprises a combination of agent-specific task queues, a centralized message buffer, a task completion status manager, and a dedicated coordinator, managing the information flow.

This implementation leverages the *threading* and *queue* libraries for concurrent agent behavior and task buffering. The key classes `SharedMemory`, `CommunicationLeader`, and `SharedCompletedSubtasks` are defined in `shared_memory_infrastructure.py`.

Shared Memory The `SharedMemory` class serves as an intermediate buffer for messages. It is designed to receive and store all incoming messages and notify of new additions. Agents are not actively connected to the structure, but rather, it allows for them to deposit final results and for the Communication Leader to retrieve messages.

The core storage constitutes a Python *list* of JSON objects (individual messages). Access to the list is protected by leveraging locks from the *threading* library to prevent race conditions caused by multiple agents attempting to access the resource. For notifying the Communication Leader of new messages, `threading.Condition` associated with the lock is used, reducing unnecessary CPU consumption. `SharedMemory` also holds an *activity* flag, representing whether the system is operational, primarily for graceful shutdown purposes. Additionally, an instance of the `SharedCompletedSubtasks` class is connected to the Shared Memory that manages the completion status of sub-tasks.

The class includes a method for writing a message into the structure, `write_message`. It acquires the lock through the internal condition, stores the input message, and crucially notifies the Communication Leader (CL) to read messages. The `read_all_messages` method (called by the CL) locks the structure as well and copies the current state of the list. After this operation, the original messages are cleared, using the `clear` method of this class, to ensure all messages are processed only once. The copied messages are then returned to CL for processing.

Communication Leader The `CommunicationLeader` class implements an active coordinator, running as a separate thread. It continuously monitors the Shared Memory for notifications of new messages through the use of the Shared Memory's condition lock.

The class is initialized with a reference to the shared message structure and references to all of the queues assigned to individual agents. CL needs these references to accurately route the messages to the corresponding agent.

After acquiring messages from the Shared Memory, CL extracts the specific message type from the JSON object and invokes the corresponding method. If a task assignment is processed, the target agent's name is also extracted from the message, allowing CL to route it accordingly. After receiving a task result, the `SharedCompletedSubtasks` instance connected to the shared memory is updated to maintain completion status for dependency management. The task results are forwarded to the AA's queue as well.

Dependency Management The `SharedCompletedSubtasks` class, instantiated inside Shared Memory for centralized management, aims to provide the necessary information about sub-task statuses for effectively handling subsequent tasks that logically follow each other. This is a fairly common scenario. For example, the Implementation Agent can be tasked with implementing a new feature (e.g., function), and the Test Agent is requested to generate a test case for this feature. Naturally, the Test Agent will have to wait for the first sub-task to complete before initiating its operation.

The initial completion information entries are constructed for each sub-task with the `initialize_subtasks` method (triggered by receiving the `subtasks_init` message), and they are stored in its internal dictionary. These entries hold the information about task dependencies, formatted as a list of dependent sub-task step numbers. This list is populated directly by AA for each sub-task when the plan is generated.

The `update_subtask` method is called by CL for every task result received. It updates the stored dictionary of entries, changing the completion status of that sub-task and appending the corresponding agent result. The dictionary is protected by a condition lock. Updating it notifies all of the specialized agents, which are waiting for dependent tasks to complete, to check if they can begin task execution.

Active specialized agents, assigned with tasks that have any dependencies, enter the `wait_for_dependencies` method, where they passively wait (using `Condition.wait()`) for all of the dependent tasks to complete, re-checking the completion status of sub-tasks whenever notified by the `update_subtask` method.

Once all dependent sub-tasks have been completed, the `get_subtask_content` method is used to extract agent results from the dictionary to incorporate relevant information in the agent's prompt.

5.8 Specialized Agents

All specialized agents within the system (Implementation, Debugging, Testing, Documentation) reuse one core class, `Agent`, defined in `shared_memory_infrastructure.py`. Its primary purpose is to use its assigned embedded LLM (`UnifiedAgent`) to autonomously solve the agent-specific sub-tasks assigned to it by CL, as part of the plan generated by AA.

Similar to the `CommunicationLeader`, this class inherits from `threading.Thread`, facilitating the concurrent and separate operation of specialized agents, which enables parallel processing, given that dependencies are resolved.

The agent initializes with a unique name (e.g., `test_agent`), a reference to the shared memory, a specialized agent prompt, and model configuration parameters. During initialization, a new `queue.Queue` is created for the agent to be later passed to CL, and an internal `UnifiedAgent` is instantiated.

Agent Lifecycle After the agent is activated, it passively waits (without consuming CPU resources) until a task object is placed in its dedicated queue by CL. Upon receiving this task, the agent invokes its internal `process_task_iteratively` method to handle task execution using the reusable iterative logic component. Once the execution completes, the result is sent back to the shared memory as a `task_result` message. After that, the agent resumes waiting for new tasks.

Task Execution This process is confined in the `process_task_iteratively` method. It first extracts relevant information from the sub-task object (sub-task assignment description, original problem statement, etc.), including the dependencies appended to the object by AA. If dependencies are present, the agent invokes the `wait_for_dependencies` method of the `SharedCompletedSubtasks` class, which blocks it until the dependent tasks complete.

The agent’s prompt is constructed from the agent-specific instruction prompt, the sub-task assignment description, the developer’s original query, the brief task context provided in the task object by AA, and the collected results of dependent tasks. These components are concatenated and passed to the core `agent_logic_loop` function defined in `agent_logic.py`, which handles the actual process of solving the task using the underlying LLM.

Result and Graceful Shutdown Result received from `agent_logic_loop` is returned to the thread’s main function. An internal `send_message` method is invoked to write the result (as a message of type `task_result`) to the shared memory. CL subsequently receives this message, forwards it to AA, and updates the `SharedCompletedSubtasks` instance for consistent dependency management.

Both the `Agent` and the `CommunicationLeader` classes incorporate a shutdown method for gracefully terminating their operation. In the case of `CommunicationLeader`, it also clears all of the existing messages in the shared memory.

5.9 Prompt Engineering

The careful process of prompt engineering is crucial in agentic AI systems. It goes beyond simple instruction to the model, encoding complex personas and behaviors for the agent’s underlying LLM.

The prompt (or a collection of prompts) given to the agent dictates everything from its role and reasoning approach to its interaction with tools and the output format.

Therefore, designing the most suitable prompt is a nuanced process that severely impacts the overall functionality of the agents. Even a light modification can lead the agent’s reasoning down an undesirable path.

Prompt Preparation Before an agent begins its operation, its initial conversation state is constructed from multiple components. The final conversation starter typically comprises:

- **System Prompt:** Shows the model function and parameter descriptions of the agent’s tools and how to interact with them. This provides the agent with information about the available functionality (like `search_code_snippets`) and the required syntax for invocation (e.g., `<execute_ipython>...</execute_ipython>`) and completion signaling (`<finish>`). Given that all agent LLMs within the system use the same tools, it is not necessary to design separate system prompts for this purpose.
- **User Prompt (Jinja2 Template):** These prompt templates are loaded and rendered by the `PromptManager` class (from the integrated `LocAgent` system). They present the model with an example use case of the type of task they are expected to complete, including exemplary tool calls, to enhance focus and output quality.

- **Task-Specific Instructions (Python Constants):** Defined in `prompts.py`, these provide the core, agent-specific workflow details and instructions necessary for task completion.
- **Problem Statement/Query:** The actual task content provided by the developer or generated by the Analyst Agent.

Agent-Specific Prompts Since agents' roles are defined strictly by their prompts, it is necessary to create a separate prompt for each. The core guidance lies in the **specialized agent prompts**. Here, the agent's role is clearly stated, detailing the expected responsibilities. A suggested workflow for the agent is provided with descriptions of what to focus on and how to progress further. The prompt is finished with a clearly defined output format and any additional notes for the reasoning process.

Key Techniques There are various methods to enhance an agent's prompt for maximizing effectiveness. Prompt in this system, for example, structure the suggested workflow into explicit steps with headings. This encourages the agent's LLM to do the same, effectively triggering explicit reasoning.

Explicitly showcasing the correct way of invoking a tool is vital for a smooth operation process. If the agent hallucinates a tool call, it can break the whole execution. This system's prompts clearly state the valid tool usage using the required `<execute_ipython>` syntax.

Sometimes, content necessary for the prompt is required dynamically during execution. Using the `Template` class of the `string` library allows the usage of variables inside the prompt to dynamically inject runtime information.

The way the overall information in the prompt is structured is also a key factor for the prompt's effectiveness. The flow should be logical and clear to reduce the possibility of confusing the LLM with inconsistent information.

Chapter 6

Results Assessment

This chapter will review the achieved results of CogniCollab as a helpful AI assistant tool. The assessment focuses on demonstrating the system’s functionalities and comparing its approach and potential outputs with existing systems.

6.1 System Demonstrations

To showcase the achieved capabilities, two example scenarios were chosen, incorporating a typical question and task assignment targeted at software development.

CogniCollab was provided with a sample codebase, which is in development, to ensure standard conditions similar to real-world project properties. This chosen project focuses on collecting and categorizing data from Wikipedia infoboxes.

The first example highlights the Analyst Agent’s clarification functionality, capable of explaining relevant code segments based on an abstract description of the requested information. Subsequently, the second example will display the results of the system’s detailed task solution process.

6.1.1 Clarification Scenario

This scenario demonstrates CogniCollab’s ability to process natural language queries requesting code functionality explanations. The objective is to show how AA, operating in its clarification mode, can understand the developer’s need for clarification and satisfy it with corresponding detailed descriptions.

The query was designed as follows:

„Describe the purpose of the 'is__water__area' static method in 'ent__waterarea.py'.
How does it use infoboxes, categories, and titles to determine if a page represents
a water area, and what do the returned 'level' and 'type' values signify?“

Upon receiving the task, AA’s mode classifier correctly detects that this objective is of type **clarification**. The explanation request is subsequently forwarded to the clarification module, which iteratively performs code searches over the specified project using the `search_code_snippets` tool. Relevant keywords and terms are isolated (such as `is__water__area` or `ent__waterarea.py`) from the query by the model to focus the searches on relevant segments of the codebase.

Once the agent understands the necessary context, it outputs the final detailed clarification, potentially enhanced by occasional code snippets (for this specific case, snippets were

deemed unnecessary by the agent). This result is propagated to the UI and presented to the user in a clear format. The final formatted response observed by the user is displayed in Figure 6.1.

```
OBSERVATION: The is_water_area static method in the ent_waterarea.py file is designed to determine if a given Wikipedia page represents a water area based on its title and content. The method uses infoboxes, categories, and titles to make this determination. It returns a tuple (level, type) where level indicates the confidence level of the identification (1 for high confidence, 2 for medium, 3 for low), and type describes the specific criterion used to identify the page as a water area.
```

Explanation: The `is_water_area` method performs several checks to determine if a page represents a water area:

- 1. Infobox Check:**
 - It searches for specific infobox templates in the page content using regular expressions.
 - If an infobox for "vodní plocha" (water area) or "moře" (sea) is found, it returns `(1, "Infobox 'vodní plocha'" or (1, "Infobox 'moře'")`.
 - If the infobox "oceány" (oceans) is found, it returns `(1, "Infobox 'oceány'")`.
- 2. Category Check:**
 - It checks for specific category tags in the page content.
 - If the page is categorized under "Rybníky" (ponds), "Jezera" (lakes), or "Říční jezera" (river lakes), it returns `(2, "Kategorie 'Rybníky'")`, `(2, "Kategorie 'Jezera'")`, or `(2, "Kategorie 'Říční jezera'")`.
- 3. Title Check:**
 - It checks the page title for specific suffixes that indicate it might be a water area.
 - If the title ends with "rybník" (pond), "jezero" (lake), "moře" (sea), "oceán" (ocean), or "tůň" (small pond), it returns `(3, "Název 'rybník'")`, `(3, "Název 'jezero'")`, `(3, "Název 'moře'")`, `(3, "Název 'oceán'")`, or `(3, "Název 'tůň'")`.

If none of these checks succeed, the method returns `(0, "")`, indicating that the page is not identified as a water area.

The `level` value in the returned tuple indicates the confidence level:

- `1`: High confidence (based on Infobox).
- `2`: Medium confidence (based on category).
- `3`: Low confidence (based on title).

The `type` value in the returned tuple describes the specific criterion used to identify the page as a water area, such as "Infobox 'vodní plocha'", "Kategorie 'Rybníky'", or "Název 'rybník'".

Figure 6.1: The final response of the Analyst Agent’s clarification mode, formatted and displayed in the Streamlit user interface.

The Analyst Agent was able to effectively explain the areas in question, enhancing the developer’s understanding of the code and generally aiding the development of the project. Notably, the user did not have to include any additional code context directly in or appended to the query. Only the local repository path was specified, and the agent performed its own exploration.

6.1.2 Completion Scenario

The completion scenario aims to illustrate CogniCollab’s end-to-end capability in handling a complex development task, formulated in an abstract natural language assignment. The assigned task requires planning, code additions, and generating documentation and test cases, showcasing the multi-agent collaboration.

The request sent to CogniCollab was:

„Enhance the data processing for the 'EntWaterArea' class ('ent_waterarea.py') to improve the traceability of continent information. The system should explicitly track and record whether the continent assigned to a water area entity was sourced directly from the primary infobox data on its Wikipedia page. This provenance information (indicating if the continent was found in the infobox)

should be included as a distinct field in the final serialized output for each water area. Please also ensure this tracking mechanism is verified for accuracy and that the new output field is clearly documented.“

While this query specifies the relevant segment of the codebase, it does not include step-by-step instructions on how to complete the task, highlighting the system’s ability to infer necessary operations from a casually written assignment.

After triggering AA’s planning module by correctly identifying the completion scenario, LocAgent was invoked. Based on the user query, LocAgent analysed the codebase and identified the following specific locations as relevant to the assigned task:

```
Based on the provided code, here are the specific locations that require modification to  
↪ resolve the issue:
```

1. ****Add a new field for continent source tracking**:**
 - Add ‘continent_source’ as an attribute in the class initialization and document it.
2. ****Modify the ‘__init__’ method to initialize the new field**:**
 - Initialize ‘continent_source’ to ‘None’.
3. ****Modify the ‘line_process_infobox’ method to capture the source of continent data**:**
 - Set ‘continent_source’ to ‘True’ when the continent is sourced from the infobox.
4. ****Modify the ‘serialize’ method to include the new field in the output**:**
 - Add ‘continent_source’ to the serialized output.
5. ****Document the new field in the class description**:**
 - Update the class docstring to include information about ‘continent_source’.

Let’s finalize the locations:

```
'''  
ent_waterarea.py  
line: 33  
class: EntWaterArea  
function: (class definition)  
# Add the new field to the class description  
# Update the docstring to include continent_source field  
  
ent_waterarea.py  
line: 45  
class: EntWaterArea  
function: __init__  
# Initialize the new field  
self.continent_source = None  
  
ent_waterarea.py  
line: 109  
class: EntWaterArea  
function: line_process_infobox  
# Set the new field when the continent is sourced from the infobox  
self.get_continent(self.del_redundant_text(rexp.group(1)))  
self.continent_source = True  
  
ent_waterarea.py  
line: 276  
class: EntWaterArea  
function: serialize  
# Add the new field to the serialized output  
self.continent,  
self.continent_source,  
'''
```

The analysis consists of a summary of necessary changes and where to implement them, followed by a structured output of relevant code objects.

Using this acquired information as context, AA then generated the following detailed plan of action:

```
[
  {
    "step": 1,
    "type": "task",
    "sender": "analyst_agent",
    "target_agent": "implementation_agent",
    "content": "Add a new field 'continent_source' to the 'EntWaterArea' class in
      ↳ 'ent_waterarea.py' to track the source of the continent data. Initialize this
      ↳ field to 'None' in the '__init__' method. Update the 'line_process_infobox' method
      ↳ to set 'continent_source' to 'True' when the continent is sourced from the
      ↳ infobox. Add 'continent_source' to the serialized output in the 'serialize'
      ↳ method.",
    "original_task": "Enhance the data processing for the 'EntWaterArea' class
      ↳ ('ent_waterarea.py') to improve the traceability of continent information.",
    "dependencies": [],
    "context": "ent_waterarea.py:EntWaterArea (__init__, line_process_infobox, serialize)"
  },
  {
    "step": 2,
    "type": "task",
    "sender": "analyst_agent",
    "target_agent": "document_agent",
    "content": "Update the class docstring in 'ent_waterarea.py' to include documentation for
      ↳ the new 'continent_source' field, describing its purpose and how it is used.",
    "original_task": "Enhance the data processing for the 'EntWaterArea' class
      ↳ ('ent_waterarea.py') to improve the traceability of continent information.",
    "dependencies": [1],
    "context": "ent_waterarea.py:EntWaterArea (class docstring)"
  },
  {
    "step": 3,
    "type": "task",
    "sender": "analyst_agent",
    "target_agent": "test_agent",
    "content": "Develop test cases to verify that the new 'continent_source' field is
      ↳ correctly set and included in the serialized output. Test scenarios should include
      ↳ cases where the continent is sourced from the infobox and cases where it is not.",
    "original_task": "Enhance the data processing for the 'EntWaterArea' class
      ↳ ('ent_waterarea.py') to improve the traceability of continent information.",
    "dependencies": [1],
    "context": "ent_waterarea.py:EntWaterArea (line_process_infobox, serialize)"
  }
]
```

The individual steps are subsequently extracted and forwarded to the **SharedMemory** component as sub-task assignments (to be distributed by the **CommunicationLeader**). After collecting the individual results from agents, they are presented back to AA. The process finishes when AA constructs the final message for the developer, pleasantly formatted with markdown tags, ensuring a readable appearance in the UI.

The system results, generated by AA, are well structured in separate markdown sections based on the sub-objective type (e.g., feature implementation, testing). Individual agents provide short explanations with their answers, so AA uses them to enrich the displayed modified code snippets with helpful descriptions.

The full final result from the system for this specific scenario can be found in the Appendix.

6.2 CogniCollab in the Context of State of the Art AI-driven systems

To position CogniCollab within the rapidly evolving landscape of AI-powered software development tools, this section compares its distinct characteristics and trade-offs with other AI-based systems aiming for similar enhancements.

The comparison will focus mainly on architectural decisions, operational principles, context awareness, and overall approach to task automation. The information in this section is based on publicly available knowledge and the design principles of CogniCollab as detailed in this thesis, and highlights the system’s unique contributions and practical orientation.

6.2.1 MetaGPT

This system employs the concept of simulating a whole software company by utilizing a team of AI-powered agents [23]. High-level user prompts are processed and transformed into complete software projects, including advanced documentation like Product Requirement Documents (PRDs). MetaGPT’s agents assume roles following the Standardized Operating Procedures (SOPs), such as an Architect or Engineer, to generate completely new projects and applications.

Comparisons and Correlations After analysing MetaGPT’s characteristics, it is apparent that CogniCollab differs primarily in the system’s operational focus.

MetaGPT specializes in interpreting a high-level *idea* and subsequently proposing the whole corresponding functionality implementation. Therefore, the system’s context is primarily aimed at the new, requested project.

Conversely, CogniCollab is more suitable for targeted modifications/additions within established codebases. Its enhanced context awareness ensures effective operation on existing code. Input can also be represented by a high-level idea, but the assignments are generally more specific, drawing the tool’s focus on the relevant code segments.

6.2.2 AutoGen

Autogen [62], a framework developed by Microsoft, is a versatile system that provides means to build custom multi-agent conversational applications. It offers the necessary building blocks for creating systems where multiple agents (represented by either LLMs, humans, or tools) work together by exchanging messages. Therefore, this framework is not itself an out-of-the-box development assistant, but rather it lets users construct their own such systems.

CogniCollab Parallels CogniCollab can be classified as a specific, opinionated application employing the basic principles that AutoGen upholds. Many of CogniCollab’s system components (such as the `UnifiedAgent` class for LLM interaction, or the communication infrastructure) could be easily built using the AutoGen framework or similar. Notably, CogniCollab offers a more assembled solution for software development. Its predefined agent roles and integrated code localization tool ensure a ready-to-use assistant, tailored for most common development scenarios, while AutoGen supplies the foundational elements the users require to design and build the specific agentic implementation.

6.2.3 Self-Organized Agents

The Self-Organized Agents (SoA) [29] framework is a state-of-the-art multi-agent system designed to address the issues that arise as the task complexity and codebase size increase. It is specifically targeted at scalable and efficient generation and optimization of large-scale code with a focus on tackling context limitations of single-agent implementations. This highly correlates with CogniCollab’s aim of enhancing context awareness and adaptation to larger projects.

SoA employs a hierarchy of „Mother“ and „Child“ agents that operate independently, yet progressively further the completion of the ultimate goal. The novelty of this framework lies in its dynamic scalability, where the Child agents multiply dynamically based on the complexity of the assignment. This allows the code quantity to increase while preserving the constant workload managed by each agent. Mother agents process tasks, decompose them, create code templates with unfinished functionality, and delegate sub-problems to the instantiated Child agents. Furthermore, Child agents can iteratively refine their code based on test results and feedback from their parent Mother agent. SoA’s multi-agent concept is displayed in Figure 6.2.

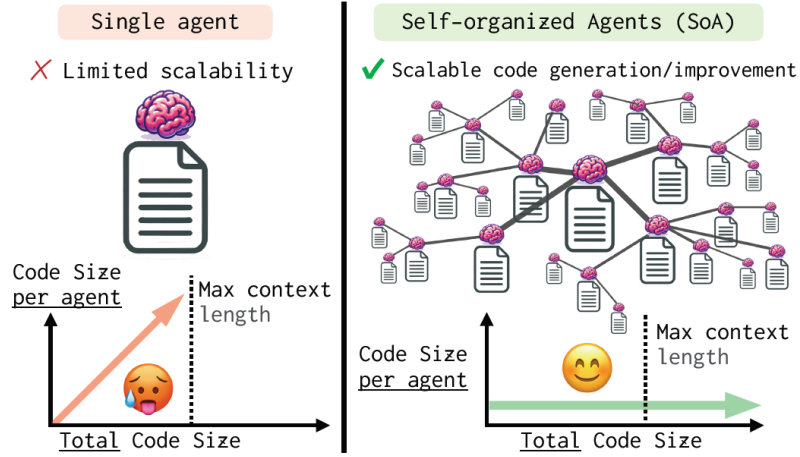


Figure 6.2: **Left:** Scenario for a single agent completing code completion tasks. As the complexity and assignment scale increase, the agent is limited by its maximum context length, preventing it from completing the task. **Right:** SoA’s dynamic agent instantiation, which evenly divides the assignment across agents, ensuring a constant workload. Figure taken from [29].

Comparison with CogniCollab While SoA focuses on generating and optimizing new code, with a highly scalable level of task complexity, and has a dynamic number of agents, CogniCollab employs a predefined set of specialized agents with various roles, supported by one central decomposer and planner. Integration with LocAgent underlines CogniCollab’s focus on enhancing existing projects by providing the ability to search over code segments. Conversely, SoA excels at building new code through hierarchical delegation and subsequent iterative, feedback-complemented improvement of the outputs. Agents in both systems are guided by predefined rules, ultimately leading to completing the assigned task, but their approach to automating software development and areas of improvement differ considerably.

6.2.4 Aider

Aider assistant¹ serves the purpose of an AI pair programmer, operating directly in the user’s command-line interface. It is specifically designed to understand and modify code within local git repositories. Natural language instructions are passed to Aider, guiding it to make context-aware changes, refactor existing code, and implement new features, furthering the developer’s local development. Aider’s workflow is usually iterative, including the user in the conversation by outputting proposed diffs and asking for confirmations before applying the changes. Users are allowed to specify relevant files in the codebase to provide project context, while Aider also builds a conceptual map of the repository, enabling it to reason over the code relationships.

Aider and CogniCollab Both of these systems focus on enhancing an established, local project, significantly reducing the time spent on typical software development tasks. Agents in both frameworks incorporate iterative reasoning logic, underlining the importance of this agentic capability. While the single-agent Aider involves the developer in the workflow, enforcing the concept of human-in-the-loop systems, and facilitates direct file changes, CogniCollab contributes by its multi-agent nature and enhanced context awareness. Aider provides strong aspirations and implications for future work on CogniCollab.

6.2.5 Summary

To conclude this section, while the field of AI-driven assistant systems employing agentic capabilities is an ever-growing field, CogniCollab attempts to establish a distinct position by combining novel characteristics of various systems to ultimately further software development automation. Systems like MetaGPT and SoA strongly demonstrate the innovative capabilities of generating completely new projects from natural language assignments and ideas, with SoA particularly addressing scalability. Frameworks such as AutoGen provide essential components and a safe environment to build custom agentic implementations, facilitating various applications with nuanced needs.

Conversely, CogniCollab and other tools like Aider focus on enhancing the already created code, performing code-aware reasoning over necessary changes and proposed additions. CogniCollab differs from typical pair-programming assistants by employing a multi-agent architecture for this purpose. Advanced context enhancement allows agents to search through extensive repositories, while a dedicated planner delegates sub-tasks to support the thoroughness of task solutions. By aiding the developer in routine code-related tasks, CogniCollab aims to streamline developers’ productivity and to showcase the innovative capabilities and applications of LLM-based systems.

6.3 System Limitations

This section will focus on critically acknowledging the various shortcomings of CogniCollab. While it can successfully aid in standard project-level development problems, this system’s development is still in its initial stages, with many areas for improvement.

¹Aider’s public repository can be found on GitHub at: <https://github.com/Aider-AI/aider>

6.3.1 Latency

The system accesses cost-free models through API keys provided by the OpenRouter platform, which have limited throughput and therefore introduce API-related latency. Calls through this platform can range in duration from a couple of seconds to even a few minutes. Since each agent performs multiple API calls (for each message in its reasoning loop), these individual delays accumulate, prolonging the overall execution process. In extreme cases, the system can take as long as 10 minutes to complete the overall task. This is a significant consideration for the tool’s usage, though it does not affect the system’s core functionality.

This issue could be mitigated by using API keys that are permitted for more inference. For example, creating an API key directly from OpenAI would let the user use powerful and fast models such as GPT-4. However, these keys usually represent a paid service, reducing the tool’s accessibility.

6.3.2 Fault Tolerance

The framework is exposed to failures in its intermediate steps, causing subsequent issues in the rest of the execution.

CogniCollab utilizes the `litellm` library, which handles retries in cases when the underlying LLM fails to respond to the API call. This functionality reduces the implementation’s complexity by removing the need for difficult manual error handling. Furthermore, there are default agent responses in place, informing AA that a specific sub-task could not be completed. This ensures that even if one agent fails, it is still possible to reach the collective objective (with minor gaps in the final solution).

However, there are cases where unexpectedly large results from the search tool can overload the single agent’s context window, preventing it from further operation. This means that the agent encounters files or code objects (e.g., classes) of significant size, which get retrieved as a whole. Projects large in number of files are still highly applicable to CogniCollab. These problematic cases are detected, and the reasoning loop is terminated with a default response. These complex issues and possible simultaneous failures of multiple agents can significantly deteriorate the quality of the framework’s final result, possibly even causing the whole system to fail. The limitation could potentially be more contained by leveraging access to premium models with larger context windows.

6.3.3 Hallucinations

CogniCollab can encounter errors stemming from the LLM outputs themselves. Since the agents autonomously request tool calls based on the context needed, it is important that the LLMs reliably conform to the specified syntax rules. Even though the prompts explicitly state this information, it is still possible for CogniCollab’s operational units to format the call incorrectly or even hallucinate non-existent tool calls. In some extreme cases, an agent might get considerably confused and attempt to execute code from the developer’s codebase.

Additionally, the LLM-driven components need to adhere to the instructions pertaining to output format and tagging the messages for processing (e.g., a finish flag signaling the final response) to avoid parsing errors or omitted results. A recurring issue is that an agent outputs the final message, but forgets to tag it. After noticing the problem, it inputs the finish tag as the sole content of the following message, resulting in an empty response. This, however, is handled by detecting such cases and using the last **valid** message (with relevant content) as the agent’s final response.

Chapter 7

Conclusion

This thesis aimed to address the ever-lasting issues caused by overloading the developer workforce with routine, yet necessary, code-related tasks (testing, debugging, basic feature additions, etc.). The primary purpose was to explore, design, and implement a multi-agent AI assistant system to mitigate these issues by automating segments of the software development process.

Agentic AI systems are in continuous development. Their capabilities, strongly resembling human workflows and reasoning processes, show great potential for software engineering automation, showcased by various already existing state-of-the-art frameworks.

In the context of these remarkable implementation approaches, CogniCollab was introduced as a representative solution, leveraging multiple Large Language Models and employing agentic principles with enhanced context awareness through system integration with the novel LocAgent.

7.1 Summary of Key Contributions

CogniCollab enforces a modular architecture with specialized components, each serving a nuanced purpose. Multiple agents with defined roles, through careful prompt engineering, collaborate by exchanging messages and task results to ultimately reach the shared goal. This concept is supported by a dedicated Analyst Agent, serving as an agent supervisor, which decomposes the complex task into elemental problems, decoupling the workload into more manageable parts. LocAgent’s provided analysis and detected intent classes from the fine-tuned CodeBERT further strengthen the Analyst Agent’s reasoning process. Specialized Agents are responsible for completing delegated sub-tasks through iterative code searching and reasoning over retrieved segments, while all communication is passed through the Shared Memory component and managed by the rule-based (without the enhancement of an LLM) Communication Leader component.

Utilizing novel technologies, such as multiple powerful LLMs, model API querying through `litellm`, Streamlit application UI, or prompt-based role definition, enables these components to understand and adapt to the assigned task, while their individual workflows remain isolated, ensuring separation of concerns. Allowing agents to freely explore the necessary context mitigates the need for manually sharing the referenced code as part of the model’s prompt. Core concepts of CogniCollab include a combination of implicit reasoning, elicited by the underlying LLM, and explicit reasoning introduced by the designed prompts and agent control structures.

CogniCollab positions itself alongside other innovative frameworks. MetaGPT and SoA represent powerful multi-agent solutions for generating completely new code on a large scale. While MetaGPT incorporates various software engineering positions (e.g., Project Manager, QA Engineer), as specified by SOPs, into the agent roles, SoA addresses task complexity by dynamically scaling the number of agents to facilitate thorough task decomposition and delegation.

Toolkits like AutoGen enable users to satisfy their diverse needs by creating custom applications from foundational agentic components.

Additionally, Aider proves to be a prominent pair-programming assistant for local development of git repositories. Oriented on furthering established projects, it includes the developer in the conversation for guidance and requests confirmations before applying modifications, proposed by displayed diffs.

While far from their level of novelty and innovation, CogniCollab represents a strong contribution to automating segments of the software development process by employing various characteristics of these revolutionary systems and applying them to a distinct area of focus. As MetaGPT and SoA, it leverages multiple agents to decompose complex tasks into manageable sub-problems. However, the team of agents is more distilled than in MetaGPT, including roles targeted at typical areas of programming expertise (such as Tester, Debugger), and the team size is constant, contrary to SoA, where the workforce size increases dynamically. As does Aider, CogniCollab aims to aid developers in expanding their local projects. The single-agent assistant offers helpful features like direct file changes or incorporating user feedback, while CogniCollab, integrated with the LocAgent system capabilities, is potentially more prominent in understanding the existing codebase.

7.2 CogniCollab Significance and Limitations Acknowledgement

As demonstrated by the example scenarios in Chapter 6, the framework exhibits refined capabilities in reasoning and planning that satisfy the most common, routine software development tasks the developers can encounter. For the completion mode case, the whole end-to-end workflow was described (from analysis through decomposition, planning, and sub-task completion), finishing with a clear and structured result, appropriately formatted in the user interface. CogniCollab’s application to software development as an assistant tool is underlined by the clarification case, where the system was able to provide a nuanced explanation of the unclear code, reducing developers’ time spent on understanding new projects during team onboarding. Together, these two functionalities provide a significant aid during typical day-to-day activities for developers.

While CogniCollab displays these proficiencies, the system, in its current stage of development, also presents certain limitations that were identified during its assessment in Chapter 6. These mostly revolve around operational latency, resulting from a combination of LLM API inference and free model access with reduced throughput. Additionally, even though the system was designed with fault tolerance mechanisms, such as library-enforced API retries and error handling or default agent responses, the agents are still susceptible to failure, primarily caused by large code search results. These edge cases endanger the completeness of the final response and can possibly even cause the whole workflow to fail (multiple agents are simultaneously unable to produce completions).

The encountered limitations attest that CogniCollab should be subjected to further development in the future in order to truly aid in software development environments.

7.3 Future Work

The development of CogniCollab has laid a strong foundation for a powerful AI assistant. However, the found limitations, unexplored potential, and observations of other novel frameworks open doors to various directions for future research and areas of enhancement.

- **Robustness and Reliability:** Issues pertaining to latency could be mitigated by exploring premium and optimized model APIs, which would increase the throughput and speed up LLM inference. Additionally, code search results could be cached, possibly reducing the overhead of searching over the codebase with each new prompt. Or, the agents could be actively connected to the shared memory pool, ensuring that the acquired context is seen by all agents concurrently.

Furthermore, CogniCollab’s fault tolerance can be improved by incorporating more sophisticated error detection mechanisms and subsequent recovery logic. For instance, agents could be better equipped to handle unexpectedly large code search results by enforcing shorter function responses or summarizing the information. To mitigate extreme cases, fallback operations handling deterioration of multiple agents should be in place, possibly incorporating a system-level retry.

- **Agent Expansion:** Further improving the already established contextual awareness of CogniCollab is necessary to progressively improve the quality of the generated code completions and modifications. The deeper the agents understand the referenced code, the better they can reason over the requested functionality. This could include refining the integration with LocAgent, utilizing other prominent code retrieval and analysis tools, or implementing a mechanism enabling agents to adapt to specific project conventions over time (for example, through a cached structure keeping the corresponding information about the project).

The Analyst Agent is a candidate for improvement as well. Enhancing its planning capabilities by allowing it to explore multiple decomposition paths [66] or to adjust its planning approach dynamically based on the feedback from specialized agents could substantially increase result quality.

- **User Interface:** The user interaction layer could be further enriched by new, useful components, ensuring an effortless user experience. For example, additional configuration fields could be added to allow the developer to choose a custom agent setup, selecting different LLMs for each agent, and passing API keys. Furthermore, the application could potentially monitor the state of the task completion process, informing the user about progress (e.g., with a progress bar).
- **Evaluation:** The system should be subjected to thorough output monitoring to achieve a deeper assessment of the framework’s state. Quantitative studies could be carried out, measuring the impact of CogniCollab on overall developer productivity, completion durations, and the quality of generated code or documentation, in comparison to, for example, manual efforts spent on the same tasks.

User studies could also prove useful, as they would potentially provide valuable feedback on the current usability, ultimately increasing the system’s effectiveness and user satisfaction.

- **New Features:** In pursuit of increasing the system’s overall usability and application, completely new features could be added to CogniCollab. Straightforward additions of new agents with specific roles are ensured and encouraged by the employed modular architecture. This could broaden CogniCollab’s specialization to include a wider range of code-oriented tasks.

Moreover, CogniCollab could adopt Aider’s novel direct file changes capabilities, reaching greater autonomy in its software development assistance. This would aim to unburden the developer of making the modifications manually, saving their precious time for higher-level assignments.

By moving forward in these directions, future versions of CogniCollab have the potential to become a vital part of the AI assistant system arsenal. Continuous and ever-improving alleviation of the workload developers encounter shifts focus towards creative design and complex architectural decisions.

Bibliography

- [1] ABID, A.; FAROOQI, M. and ZOU, J. Persistent anti-muslim bias in large language models. In: *Proceedings of the 2021 AAAI/ACM Conference on AI, Ethics, and Society*. 2021, p. 298–306.
- [2] ACHARYA, D. B.; KUPPAN, K. and DIVYA, B. Agentic AI: Autonomous Intelligence for Complex Goals—A Comprehensive Survey. *IEEE Access*. IEEE, 2025.
- [3] ACHIAM, J.; ADLER, S.; AGARWAL, S.; AHMAD, L.; AKKAYA, I. et al. Gpt-4 technical report. *ArXiv preprint arXiv:2303.08774*, 2023.
- [4] AGHAJANYAN, A.; GUPTA, A.; SHRIVASTAVA, A.; CHEN, X.; ZETTLEMOYER, L. et al. Muppet: Massive multi-task representations with pre-finetuning. *ArXiv preprint arXiv:2101.11038*, 2021.
- [5] ALKAMLI, S. and ALABDULJABBAR, R. Understanding privacy concerns in ChatGPT: A data-driven approach with LDA topic modeling. *Heliyon*. Elsevier, 2024, vol. 10, no. 20.
- [6] ALLAMANIS, M.; BARR, E. T.; BIRD, C. and SUTTON, C. Learning natural coding conventions. In: *Proceedings of the 22nd acm sigsoft international symposium on foundations of software engineering*. 2014, p. 281–293.
- [7] BAHDANAU, D.; CHO, K. and BENGIO, Y. Neural machine translation by jointly learning to align and translate. *ArXiv preprint arXiv:1409.0473*, 2014.
- [8] BOLLE, M. D. AI’s carbon footprint appears likely to be alarming. *Peterson Institute for International Economics*, 2024.
- [9] BROOKS JR, F. P. *The mythical man-month: essays on software engineering*. Pearson Education, 1995.
- [10] BROWN, T.; MANN, B.; RYDER, N.; SUBBIAH, M.; KAPLAN, J. D. et al. Language models are few-shot learners. *Advances in neural information processing systems*, 2020, vol. 33, p. 1877–1901.
- [11] CHAKRABORTY, S.; AHMED, T.; DING, Y.; DEVANBU, P. T. and RAY, B. Natgen: generative pre-training by “naturalizing” source code. In: *Proceedings of the 30th ACM joint european software engineering conference and symposium on the foundations of software engineering*. 2022, p. 18–30.
- [12] CHEN, Z.; TANG, X.; DENG, G.; WU, F.; WU, J. et al. Locagent: Graph-guided llm agents for code localization. *ArXiv preprint arXiv:2503.09089*, 2025.

- [13] CHO, K.; VAN MERRIËNBOER, B.; GULCEHRE, C.; BAHDANAU, D.; BOUGARES, F. et al. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *ArXiv preprint arXiv:1406.1078*, 2014.
- [14] CHOWDHERY, A.; NARANG, S.; DEVLIN, J.; BOSMA, M.; MISHRA, G. et al. *PaLM: Scaling Language Modeling with Pathways*. 2022. Available at: <https://arxiv.org/abs/2204.02311>.
- [15] DEVLIN, J.; CHANG, M.-W.; LEE, K. and TOUTANOVA, K. Bert: Pre-training of deep bidirectional transformers for language understanding. In: *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 2019, p. 4171–4186.
- [16] DONG, Y.; JIANG, X.; JIN, Z. and LI, G. Self-collaboration code generation via chatgpt. *ACM Transactions on Software Engineering and Methodology*. ACM New York, NY, 2024, vol. 33, no. 7, p. 1–38.
- [17] ELMAN, J. L. Finding structure in time. *Cognitive science*. Wiley Online Library, 1990, vol. 14, no. 2, p. 179–211.
- [18] FRASER, G. and ARCURI, A. Evosuite: automatic test suite generation for object-oriented software. In: *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*. 2011, p. 416–419.
- [19] GENG, M.; PATERIA, S.; SUBAGDJA, B. and TAN, A.-H. HiSOMA: A hierarchical multi-agent model integrating self-organizing neural networks with multi-agent deep reinforcement learning. *Expert Systems with Applications*. Elsevier, 2024, vol. 252, p. 124117.
- [20] GUO, T.; CHEN, X.; WANG, Y.; CHANG, R.; PEI, S. et al. Large language model based multi-agents: A survey of progress and challenges. *ArXiv preprint arXiv:2402.01680*, 2024.
- [21] HAIDUC, S.; APONTE, J.; MORENO, L. and MARCUS, A. On the use of automated text summarization techniques for summarizing source code. In: IEEE. *2010 17th Working conference on reverse engineering*. 2010, p. 35–44.
- [22] HOCHREITER, S. and SCHMIDHUBER, J. Long short-term memory. *Neural computation*. MIT press, 1997, vol. 9, no. 8, p. 1735–1780.
- [23] HONG, S.; ZHUGE, M.; CHEN, J.; ZHENG, X.; CHENG, Y. et al. Metagpt: Meta programming for a multi-agent collaborative framework, 2023. URL <https://arxiv.org/abs/2308.00352>.
- [24] HORNIK, K.; STINCHCOMBE, M. and WHITE, H. Multilayer feedforward networks are universal approximators. *Neural networks*. Elsevier, 1989, vol. 2, no. 5, p. 359–366.
- [25] HUANG, J. tse; ZHOU, J.; JIN, T.; ZHOU, X.; CHEN, Z. et al. *On the Resilience of LLM-Based Multi-Agent Collaboration with Faulty Agents*. 2025. Available at: <https://arxiv.org/abs/2408.00989>.

- [26] HUANG, L.; YU, W.; MA, W.; ZHONG, W.; FENG, Z. et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*. ACM New York, NY, 2025, vol. 43, no. 2, p. 1–55.
- [27] HUANG, X.; LIU, W.; CHEN, X.; WANG, X.; WANG, H. et al. Understanding the planning of LLM agents: A survey. *ArXiv preprint arXiv:2402.02716*, 2024.
- [28] HUGHES, L.; DWIVEDI, Y. K.; MALIK, T.; SHAWOSH, M.; ALBASHRAWI, M. A. et al. AI Agents and Agentic Systems: A Multi-Expert Analysis. *Journal of Computer Information Systems*. Taylor & Francis, 2025, p. 1–29.
- [29] ISHIBASHI, Y. and NISHIMURA, Y. Self-organized agents: A llm multi-agent framework toward ultra large-scale code generation and optimization. *ArXiv preprint arXiv:2404.02183*, 2024.
- [30] JAIN, S.; DORA, A.; SAM, K. S. and SINGH, P. Llm agents improve semantic code search. *ArXiv preprint arXiv:2408.11058*, 2024.
- [31] JIN, H.; HUANG, L.; CAI, H.; YAN, J.; LI, B. et al. From llms to llm-based agents for software engineering: A survey of current, challenges and future. *ArXiv preprint arXiv:2408.02479*, 2024.
- [32] JORDAN, M. I. Serial order: A parallel distributed processing approach. In: *Advances in psychology*. Elsevier, 1997, vol. 121, p. 471–495.
- [33] KAPLAN, J.; MCCANDLISH, S.; HENIGHAN, T.; BROWN, T. B.; CHESSE, B. et al. Scaling laws for neural language models. *ArXiv preprint arXiv:2001.08361*, 2020.
- [34] KATZ, D. M.; BOMMARITO, M. J.; GAO, S. and ARREDONDO, P. Gpt-4 passes the bar exam. *Philosophical Transactions of the Royal Society A*. The Royal Society, 2024, vol. 382, no. 2270, p. 20230254.
- [35] KHAN, S.; NASEER, M.; HAYAT, M.; ZAMIR, S. W.; KHAN, F. S. et al. Transformers in vision: A survey. *ACM computing surveys (CSUR)*. ACM New York, NY, 2022, vol. 54, 10s, p. 1–41.
- [36] KUMAR, D. and MISHRA, K. K. The impacts of test automation on software’s cost, quality and time to market. *Procedia Computer Science*. Elsevier, 2016, vol. 79, p. 8–15.
- [37] LEWIS, M.; LIU, Y.; GOYAL, N.; GHAZVININEJAD, M.; MOHAMED, A. et al. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *ArXiv preprint arXiv:1910.13461*, 2019.
- [38] LIN, S.; HILTON, J. and EVANS, O. Truthfulqa: Measuring how models mimic human falsehoods. *ArXiv preprint arXiv:2109.07958*, 2021.
- [39] LIPTON, Z. C.; BERKOWITZ, J. and ELKAN, C. A critical review of recurrent neural networks for sequence learning. *ArXiv preprint arXiv:1506.00019*, 2015.
- [40] LIU, P.; YUAN, W.; FU, J.; JIANG, Z.; HAYASHI, H. et al. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM computing surveys*. ACM New York, NY, 2023, vol. 55, no. 9, p. 1–35.

- [41] MADAAN, A.; TANDON, N.; GUPTA, P.; HALLINAN, S.; GAO, L. et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 2023, vol. 36, p. 46534–46594.
- [42] MANDAL, A. BRIDGE: a model for modern software development process to cater the present software crisis. In: IEEE. *2009 IEEE International Advance Computing Conference*. 2009, p. 1617–1623.
- [43] N, D. G.; G, M. V. and A, V. T. Intent Classification using BERT for Chatbot application pertaining to Customer Oriented Services. In: EAI, December 2021.
- [44] NIU, C.; LI, C.; LUO, B. and NG, V. Deep learning meets software engineering: A survey on pre-trained models of source code. *ArXiv preprint arXiv:2205.11739*, 2022.
- [45] OUYANG, L.; WU, J.; JIANG, X.; ALMEIDA, D.; WAINWRIGHT, C. et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 2022, vol. 35, p. 27730–27744.
- [46] PASCANU, R.; MIKOLOV, T. and BENGIO, Y. On the difficulty of training recurrent neural networks. In: Pmlr. *International conference on machine learning*. 2013, p. 1310–1318.
- [47] PENG, S.; KALLIAMVAKOU, E.; CIHON, P. and DEMIRER, M. The impact of ai on developer productivity: Evidence from github copilot. *ArXiv preprint arXiv:2302.06590*, 2023.
- [48] PLAAT, A.; DUIJN, M. van; STEIN, N. van; PREUSS, M.; PUTTEN, P. van der et al. Agentic large language models, a survey. *ArXiv preprint arXiv:2503.23037*, 2025.
- [49] QU, C.; DAI, S.; WEI, X.; CAI, H.; WANG, S. et al. Tool learning with large language models: A survey. *Frontiers of Computer Science*. Springer, 2025, vol. 19, no. 8, p. 198343.
- [50] RADFORD, A.; NARASIMHAN, K.; SALIMANS, T.; SUTSKEVER, I. et al. Improving language understanding by generative pre-training. San Francisco, CA, USA, 2018.
- [51] RAFFEL, C.; SHAZEER, N.; ROBERTS, A.; LEE, K.; NARANG, S. et al. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 2020, vol. 21, no. 140, p. 1–67.
- [52] SCHICK, T.; DWIVEDI YU, J.; DESSÌ, R.; RAILEANU, R.; LOMELI, M. et al. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 2023, vol. 36, p. 68539–68551.
- [53] SHEN, Y.; SONG, K.; TAN, X.; LI, D.; LU, W. et al. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems*, 2023, vol. 36, p. 38154–38180.
- [54] SHINN, N.; CASSANO, F.; GOPINATH, A.; NARASIMHAN, K. and YAO, S. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 2023, vol. 36, p. 8634–8652.

- [55] SINGH, A.; EHTESHAM, A.; KUMAR, S. and KHOEI, T. T. Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG. *ArXiv preprint arXiv:2501.09136*, 2025.
- [56] SYPHERD, C. and BELLE, V. Practical Considerations for Agentic LLM Systems. *ArXiv preprint arXiv:2412.04093*, 2024.
- [57] TORNHILL, A. and BORG, M. Code red: the business impact of code quality-a quantitative study of 39 proprietary production codebases. In: *Proceedings of the International Conference on Technical Debt*. 2022, p. 11–20.
- [58] TRAN, K.-T.; DAO, D.; NGUYEN, M.-D.; PHAM, Q.-V.; O’SULLIVAN, B. et al. Multi-Agent Collaboration Mechanisms: A Survey of LLMs. *ArXiv preprint arXiv:2501.06322*, 2025.
- [59] VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L. et al. Attention is All you Need. In: GUYON, I.; LUXBURG, U. V.; BENGIO, S.; WALLACH, H.; FERGUS, R. et al., ed. *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2017, vol. 30. Available at: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- [60] WANG, L.; MA, C.; FENG, X.; ZHANG, Z.; YANG, H. et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*. Springer, 2024, vol. 18, no. 6, p. 186345.
- [61] WEI, J.; WANG, X.; SCHUURMANS, D.; BOSMA, M.; XIA, F. et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 2022, vol. 35, p. 24824–24837.
- [62] WU, Q.; BANSAL, G.; ZHANG, J.; WU, Y.; LI, B. et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *ArXiv preprint arXiv:2308.08155*, 2023.
- [63] XI, Z.; CHEN, W.; GUO, X.; HE, W.; DING, Y. et al. *The Rise and Potential of Large Language Model Based Agents: A Survey*. 2023.
- [64] XIE, C.; JIAO, M.; GU, X. and SHEN, B. Empowering AI to Generate Better AI Code: Guided Generation of Deep Learning Projects with LLMs. *ArXiv preprint arXiv:2504.15080*, 2025.
- [65] XIONG, H.; WANG, Z.; LI, X.; BIAN, J.; XIE, Z. et al. Converging paradigms: The synergy of symbolic and connectionist ai in llm-empowered autonomous agents. *ArXiv preprint arXiv:2407.08516*, 2024.
- [66] YAO, S.; YU, D.; ZHAO, J.; SHAFRAN, I.; GRIFFITHS, T. et al. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 2023, vol. 36, p. 11809–11822.
- [67] YAO, S.; ZHAO, J.; YU, D.; DU, N.; SHAFRAN, I. et al. React: Synergizing reasoning and acting in language models. In: *International Conference on Learning Representations (ICLR)*. 2023.

- [68] ZHANG, Q.; FANG, C.; MA, Y.; SUN, W. and CHEN, Z. A survey of learning-based automated program repair. *ACM Transactions on Software Engineering and Methodology*. ACM New York, NY, 2023, vol. 33, no. 2, p. 1–69.
- [69] ZHANG, S.; DONG, L.; LI, X.; ZHANG, S.; SUN, X. et al. Instruction tuning for large language models: A survey. *ArXiv preprint arXiv:2308.10792*, 2023.

Appendix A

Contents of the included storage media

```
implementation/
  analyst_agent/
    agent.py
    clarification_module.py
    mode_determination.py
    planning_module.py
  LocAgent/
    ... (Files of the integrated LocAgent system)
  lib/
    tk_dir_selector.py
  loc_agent_outputs_resources/
    ... (Folder for storing LocAgent's resources and outputs)
  prompts/
    prompts.py
  train/
    finetune_codebert.py
  workflows/
    agent_logic.py
    shared_mem_com.py
    unified_agent_class.py
  app.py
  requirements.txt
  README.md
thesis/
  ... (LaTeX source files for this documentation)
poster.pdf
xkozan01-Code_Gen_Textual.pdf
```

Appendix B

System output and Prompt example

Analyst Agent's completion mode output

```
## Original Developer Assignment
Enhance the data processing for the 'EntWaterArea' class ('ent_waterarea.py') to improve the
↳ traceability of continent information. The system should explicitly track and record
↳ whether the continent assigned to a water area entity was sourced directly from the
↳ primary infobox data on its Wikipedia page. This provenance information (indicating if
↳ the continent was found in the infobox) should be included as a distinct field in the
↳ final serialized output for each water area. Please also ensure this tracking
↳ mechanism is verified for accuracy and that the new output field is clearly documented.

## Changes Made

### 1. Implementation
The 'EntWaterArea' class has been updated to include a new field 'continent_source' to track
↳ the source of the continent data. The changes made are as follows:

- **Add 'continent_source' field to 'EntWaterArea' class**:  

  - Initialize 'continent_source' to 'None' in the '__init__' method.  

  - Set 'continent_source' to 'True' when the continent is sourced from the infobox in the  

    ↳ 'line_process_infobox' method.  

  - Include 'continent_source' in the serialized output in the 'serialize' method.

'''python
class EntWaterArea(EntCore):
    """
    Třída určená pro vodní plochy.

    Instanční atributy:
    title - název vodní plochy (str)
    prefix - prefix entity (str)
    eid - ID entity (str)
    link - odkaz na Wikipedii (str)
    aliases - alternativní pojmenování vodní plochy (str)
    description - stručný popis vodní plochy (str)
    images - absolutní cesty k obrázkům Wikimedia Commons (str)

    area - rozloha vodní plochy v kilometrech čtverečních (str)
    continent - světadíl, na kterém se vodní plocha nachází (str)
    continent_source - zdroj informace o světadílu; indikuje, zda byla informace o světadílu
        ↳ získána z infoboxu (bool)
    """

    def __init__(self, title, prefix, link, redirects, langmap):
        super(EntWaterArea, self).__init__(title, prefix, link, redirects, langmap)
```



```

- **Unit Tests**:
- **'test_continent_source_from_infobox'**: Tests that the 'continent_source' is set to
  ↳ 'True' after processing a line with the continent information.
- **'test_continent_source_not_present'**: Tests that the 'continent_source' remains 'None'
  ↳ when no continent information is found.
- **'test_serialized_output_with_continent_source'**: Tests that the serialized output
  ↳ correctly includes the 'continent_source' when it is 'True'.
- **'test_serialized_output_without_continent_source'**: Tests that the serialized output
  ↳ correctly reflects the absence of the 'continent_source' when it is 'None'.

'''python
import unittest
from unittest.mock import MagicMock

class EntCore:
    def __init__(self, title, prefix, link, redirects, langmap):
        self.title = title
        self.prefix = prefix
        self.link = link
        self.redirects = redirects
        self.langmap = langmap

    def get_aliases(self, text):
        return set(text.split(","))

    def del_redundant_text(self, text, separator=" "):
        return text.strip()

    def serialize_aliases(self):
        return ""

class EntWaterArea(EntCore):
    def __init__(self, title, prefix, link, redirects, langmap):
        super(EntWaterArea, self).__init__(title, prefix, link, redirects, langmap)

        self.area = ""
        self.continent = ""
        self.continent_source = None
        self.latitude = ""
        self.longitude = ""
        self.aliases_infobox = set()
        self.description = ""
        self.images = ""
        self.original_title = ""

    def get_continent(self, continent):
        self.continent = continent

    def line_process_infobox(self, ln, is_infobox_block):
        rexp = re.search(r"světadíl\s*=(?!)\s*(.*)", ln, re.I)
        if rexp and rexp.group(1):
            self.get_continent(self.del_redundant_text(rexp.group(1)))
            self.continent_source = True # Set the source of the continent

    def serialize(self):
        return "\t".join(
            [
                self.eid,
                self.prefix,
                self.title,
                self.serialize_aliases(),
                "|".join(self.redirects),
                self.description,
                self.original_title,
                self.images,
                self.link,
                self.continent,
                self.latitude,

```



```

        self.longitude,
        self.area,
        str(self.continent_source) if self.continent_source is not None else ""
    ]
)

class TestEntWaterArea(unittest.TestCase):
    def setUp(self):
        self.instance = EntWaterArea(
            title="Test Water Area",
            prefix="WaterArea",
            link="https://example.com/wiki/Test_Water_Area",
            redirects={},
            langmap={"en": "English"}
        )

    def test_continent_source_from_infobox(self):
        line = "světadíl = Europe"
        self.instance.line_process_infobox(line, True)
        self.assertTrue(self.instance.continent_source)

    def test_continent_source_not_present(self):
        line = "název = Test Water Area"
        self.instance.line_process_infobox(line, True)
        self.assertIsNone(self.instance.continent_source)

    def test_serialized_output_with_continent_source(self):
        self.instance.continent_source = True
        self.instance.continent = "Europe"
        self.instance.eid = "W001"
        self.instance.prefix = "WaterArea"
        self.instance.title = "Test Water Area"
        self.instance.redirects = []
        self.instance.description = "A test water area for testing."
        self.instance.original_title = "Test Water Area"
        self.instance.images = ""
        self.instance.link = "https://example.com/wiki/Test_Water_Area"
        self.instance.latitude = "48.75"
        self.instance.longitude = "16.38"
        self.instance.area = "100.0"
        expected_output = "\t".join([
            "W001", # eid
            "WaterArea", # prefix
            "Test Water Area", # title
            "", # aliases
            "", # redirects
            "A test water area for testing.", # description
            "Test Water Area", # original_title
            "", # images
            "https://example.com/wiki/Test_Water_Area", # link
            "Europe", # continent
            "48.75", # latitude
            "16.38", # longitude
            "100.0", # area
            "True" # continent_source
        ])
        self.assertEqual(self.instance.serialize(), expected_output)

    def test_serialized_output_without_continent_source(self):
        self.instance.continent_source = None
        self.instance.continent = "Europe"
        self.instance.eid = "W001"
        self.instance.prefix = "WaterArea"
        self.instance.title = "Test Water Area"
        self.instance.redirects = []
        self.instance.description = "A test water area for testing."
        self.instance.original_title = "Test Water Area"
        self.instance.images = ""

```

```

self.instance.link = "https://example.com/wiki/Test_Water_Area"
self.instance.latitude = "48.75"
self.instance.longitude = "16.38"
self.instance.area = "100.0"
expected_output = "\t".join([
    "W001", # eid
    "WaterArea", # prefix
    "Test Water Area", # title
    "", # aliases
    "", # redirects
    "A test water area for testing.", # description
    "Test Water Area", # original_title
    "", # images
    "https://example.com/wiki/Test_Water_Area", # link
    "Europe", # continent
    "48.75", # latitude
    "16.38", # longitude
    "100.0", # area
    "" # continent_source
])
self.assertEqual(self.instance.serialize(), expected_output)

if __name__ == '__main__':
    unittest.main()
'''

## Summary
The 'EntWaterArea' class has been enhanced to include a new field 'continent_source' that
↳ tracks the source of the continent data. The field is set to 'True' when the continent
↳ is sourced from the infobox, and it is included in the serialized output. The changes
↳ have been documented in the class docstring, and unit tests have been added to verify
↳ the correctness of the implementation.

```

Example of a role definition through prompts

The detailed agent prompt of the Implementation Agent was selected to showcase role definition through prompt engineering.

```

You are an expert software engineer tasked with implementing modifications and additions as
↳ part of an enhancement. Your goal is to complete the task by making the necessary
↳ code changes directly in the codebase.

Follow these steps:

## Step 1: Understand the Developer's Request
- Analyze the modification request and determine what changes need to be made in the codebase.
- Identify the file, function, or class that needs to be modified.
- Use the provided search_code_snippets tool with <execute_ipython> to locate the relevant
  ↳ code segments.
- Only use <execute_ipython> for the search tool, DO NOT try to execute any other code!

## Step 2: Make the Code Changes
- Apply the required modifications directly to the code.
- Ensure the change is correctly implemented and integrates seamlessly with the existing
  ↳ codebase.
- If the task involves adding features like logging, refactoring, or new methods, ensure the
  ↳ changes are well-structured and functional.

## Step 3: Output the Result
- Once the modification is complete, provide the updated code with all necessary changes.
- The code should be correctly modified as per the request with no further steps required from
  ↳ the analyst.

Example output structure:
[Short_explanations_of_changes_or_additions]

```

```

'''python
[Your_new_or_modified_code]
'''
<finish></finish>

- Remember to check for any dependencies or related files that might also need updates. Search
  ↳ for broader context if necessary.
- Please remember that you have to ask for code context yourself using this notation:
<execute_ipython>
# terms that are relevant for the assignment
terms = ["EntWaterArea", "get_area"]
search_code_snippets(search_terms=terms)
</execute_ipython>

! <function=search_code_snippets> notation for searching code snippets IS NOT supported !
DO NOT try to execute any of the observed code using the <execute_ipython> tags. Use them ONLY
  ↳ for searching code snippets.

IMPORTANT: When using the search tool, ALWAYS include the COMPLETE command with both the
  ↳ <execute_ipython> tags AND the search_code_snippets() function call. NEVER output a
  ↳ partial search command or just the search terms alone. If your search doesn't find the
  ↳ right files, try a new COMPLETE search with different terms.
- Focus only on your assigned task. Provide at least some explanations for your changes.

CRITICAL: Always append <finish></finish> directly to your final complete response - never
  ↳ send this tag in a separate message.

Begin!

```